

Energy Measurement of Compressed Deep Learning Models: A Literature Review

Shohinur Pervez Shohan

Independent Researcher (B.Sc. in Computer Science & Engineering, Rangamati Science and Technology University, Bangladesh)

shohan414@gmail.com

September 2026

Abstract

Model compression, quantization, pruning, and knowledge distillation, is widely adopted on the premise that a smaller model costs less energy to run. This review examines the measurement layer beneath that premise and finds it less settled than its adoption suggests. Drawing on a purposively assembled corpus of 53 works (2019–2026), supplemented by a logged literature search that tested six specific absence claims, the review traces why compression's energy benefit is not guaranteed: software energy estimators and hardware counters disagree by margins comparable to the savings compression claims to deliver; floating-point operation counts correlate weakly with measured energy once memory bandwidth, batch size, and backend kernel realization are taken into account; and compression rankings established on one hardware platform do not reliably transfer to another. A controlled evaluation would need to hold the measurement instrument, workload, and protocol fixed while compression state and platform vary, and no study in the corpus does so. The review's central contribution is not the observation that compression is unpredictable, but an account of the mechanisms — memory-bound execution, batch-size regime, and backend realization — that explain when and why it fails, together with a research agenda built around three of the resulting gaps: measurement-tool validation on compressed models, the nominal-versus-realized compression gap, and cross-platform rank-order preservation under a matched protocol.

Keywords: Green AI; model compression; energy measurement; quantization; pruning; FLOPs; hardware performance counters; edge deployment; benchmarking

1 Introduction

1.1 Motivation: the energy cost of modern artificial intelligence

Artificial intelligence has grown from a research method into a global computing workload. Schwartz, Dodge, Smith, and Etzioni documented the pattern that first pushed the field to examine its own footprint: computations used in deep learning research were doubling every few months, an estimated 300,000-fold increase between 2012 and 2018 (Schwartz et al., 2019). Later work confirms that the trajectory has continued. Training a single large model such as MegatronLM consumes enough electricity to power three American households for a year, and the compute needed to reach state-of-the-art results has doubled roughly every 3.4 months (Rajput et al., 2023).

The environmental and financial stakes of this growth now form a research area in their own right. A recent systematic review organizes green machine learning around five concerns: algorithmic efficiency, energy optimization, hardware efficiency, carbon-footprint tracking, and green benchmarks, and connects each of them to sustainability goals (Santos et al., 2026). The same review reports that standardized reporting metrics remain absent and that the trade-off between energy savings and predictive accuracy is difficult to reproduce across studies (Santos et al., 2026). Energy has become a quantity the field agrees matters, without agreeing on how to observe it.

Model compression has become the default response to this pressure. Quantization, pruning, and knowledge distillation each promise a smaller or cheaper model, and the practical shorthand that follows is simple: a lighter model costs less energy to run. Deployment toolchains, hardware vendors, and energy-aware design guidelines repeat this claim, and it shapes real engineering decisions about which model to ship and where to run it.

The empirical support for the shorthand is thinner than its adoption suggests. Fischer ground-truthed two widely used estimation tools against external hardware meters and found that dynamic estimation with CodeCarbon consistently underestimated power draw by 15 to 25 percent, while static calculators produced errors ranging from roughly minus 40 to plus 40 percent (Fischer, 2026). The measurement layer beneath the compression claim carries uncertainty of a magnitude comparable to the savings the compression is meant to deliver. Alizadeh and Castor, measuring inference across PyTorch, TensorFlow, MXNet and ONNX runtimes on CPU and GPU, found that energy and wall-clock time track each other closely (Spearman $\rho = 0.99$), which makes latency a convenient stand-in for energy and simultaneously discards the hardware- and runtime-specific factors that the proxy conceals (Alizadeh & Castor, 2024).

The granularity of measurement compounds the problem. Rajput and colleagues instrumented TensorFlow at the level of individual API calls and showed that a single call such as model fitting can dominate a program's energy budget, while the surrounding project consumes a further share outside the measured library (Rajput et al., 2023). Whole-program accounting averages over exactly the hot spots where compression changes behavior, and fine-grained accounting is not yet standard practice.

Compression research has begun to systematize this trade-off directly. Iyer, Akshatha and Suma frame quantization and pruning as choices in a joint energy-, latency-, memory-, and carbon-efficiency space, profiling both techniques across nine architectures in 276 experiments to recommend a configuration for a stated objective rather than treating accuracy as the only axis worth optimizing (Iyer et al., 2026). Their framing anticipates the scope this review adopts: a compression technique's efficiency claim is a multi-objective statement, and energy is one objective among several that a compression choice can trade against.

1.2 Scope and questions of this review

This review examines how the energy of compressed deep learning models is measured and how those models behave once deployed. It covers three strands: the methodology and tools used to measure energy, the relationship between proxy metrics such as FLOPs and measured energy, and the behavior of compressed models across hardware platforms. Training energy, carbon-accounting frameworks, and hyperparameter optimization fall outside its scope, except where they bear directly on inference-time energy claims.

The review is organized around a single tension: whether lighter models actually consume less energy in practice, and under which conditions that expectation holds or fails. Three questions give the review its shape. First, how reliable are the tools currently used to estimate the energy of compressed models, and what do validation studies show about their accuracy? Second, does a reduction in computation, measured in FLOPs or parameter count, translate into a proportional reduction in energy, or does the relationship break down once memory access and data movement are taken into account? Third, does the ranking of compression strategies by energy efficiency remain stable across hardware platforms, from server CPUs and GPUs to mobile and edge devices? Each question corresponds to one of the gaps set out in Section 7.2 (Gaps 1, 2, and 3), which the literature identifies but does not close.

1.3 Roadmap

The remainder of the review is organized as follows. Section 1.4 describes how the corpus was assembled and defines the scope of every absence claim the review makes. Section 2 establishes how energy is measured, surveying software estimators, hardware performance counters, ground-truth validation against external meters, and the standardization gap; Table 2 compares the measurement tools and Table 3 summarizes the primary empirical studies. Section 3 addresses the FLOPs-energy proxy, explaining why the assumption of proportionality fails and identifying memory bandwidth and batch size as the mechanisms that displace it. Section 4 reviews model compression techniques, covering quantization, pruning, distillation, and their combinations, and examines the accuracy-efficiency frontier they define. Section 5 turns to cross-platform behavior and asks whether efficiency rankings carry across x86 CPUs, ARM processors, GPUs, and emerging accelerators. Section 6 considers deployment conditions that short benchmarks do not reproduce: thermal throttling, battery discharge, and idle-baseline handling. Section 7 synthesizes the contradictions,

maps the resulting gaps to the evidence that exposes them (Table 4), and proposes a minimum reporting checklist (Box 1). Section 8 sets out a research agenda for the three gaps that a single controlled study can address, and Section 9 concludes.

1.4 Review Methodology

This is a narrative, argument-driven review rather than a systematic review, and its methodology is reported so that the scope of its claims can be judged. The core corpus was assembled purposively over the course of the project, through topic-driven reading and targeted retrieval of works on energy measurement, proxy metrics, model compression, and deployment platforms. That assembly was not logged as a reproducible database search, and it is described here as a purposive corpus rather than reported with search strings that were never used.

Candidate works were screened against five inclusion criteria: the work concerns the energy of machine learning models through measurement, estimation, or compression with an efficiency dimension; it reports original empirical measurements, a measurement or estimation method, or a survey or position directly relevant to these; it was published or posted between 2019 and 2026; it is written in English; and its full text was available for verification. Works concerned only with training energy and without bearing on inference-time claims, works on accuracy, calibration, or robustness without an energy dimension, and works outside machine learning systems were excluded. Of 61 candidate scholarly documents, two were duplicate copies of included works and six were excluded on scope: two training-energy-only studies, one calibration and robustness study, one pruning and quantization survey without energy data, one statistical-testing toolkit, and one survey from an unrelated application domain. Fifty-three works were included.

Throughout this review, “the corpus” denotes these 53 works, which are the entries in the reference list. The corpus is recent: 44 of the 53 works date from 2024 to 2026, with one from 2019, two from 2021, one from 2022, and five from 2023. Thirty-five are cited in preprint form, fourteen are journal articles, and four are conference or workshop papers; because several of the preprints have since appeared in peer-reviewed venues, the preprint share overstates how much of the evidence is unreviewed. Every numeric claim attributed to a corpus work was checked against that work’s full text.

Because the review’s central claims are claims of absence, a supplementary search was run on 14 September 2026 to test them against literature outside the corpus. Seven Boolean queries, one for each of the six gaps in Section 7.2 and one for the Pareto-frontier claim in Section 4.5, were run in the scite.ai citation index with a publication window of 2019 to 2026 (Table 1). The queries returned between 44 and 639 records each. The fifteen highest-ranked records per query, 105 in total, were screened at title and abstract; eleven were read at abstract level as possible counter-evidence, and one whose abstract did not settle the question was read in full. No screened record reported measured-energy evidence that closes any of the six gaps. One claim was narrowed as a result: the search located a formal accuracy-efficiency Pareto benchmark for compressed mobile classifiers whose efficiency axes are model size and latency rather than energy, so Section 4.5 restricts its claim to frontiers over measured energy.

Table 1. Supplementary search used to test the review’s absence claims.

Absence claim tested	Query string (scite.ai, 2019–2026, run 14 September 2026)	Records	Screened
Gap 1: tool validation on compressed models	("CodeCarbon" OR "RAPL" OR "software energy estimator") AND ("quantization" OR "pruning") AND ("power meter" OR "external meter" OR "ground truth")	69	15
Gap 2: nominal vs. realized compression	("pruning" OR "quantization") AND ("sparse kernel" OR "dequantization" OR "inference backend") AND "energy consumption" AND "inference"	250	15
Gap 3: cross-platform ranking	("model compression" OR "quantization" OR "pruning") AND "energy" AND ("cross-platform" OR "multiple hardware platforms" OR "heterogeneous hardware") AND ("ranking" OR "rank order")	528	15
Gap 4: on-device (PMIC) validation	("PMIC" OR "power management integrated circuit" OR "fuel gauge") AND "smartphone" AND ("external power meter" OR "validation") AND ("deep learning" OR "neural network")	44	15
Gap 5: battery state and compression	("battery state of charge" OR "battery discharge") AND ("quantization" OR "pruning" OR "model compression") AND ("inference" OR "energy consumption")	284	15
Gap 6: thermal behavior and compression	"thermal throttling" AND ("quantization" OR "pruning" OR "model compression") AND "energy"	168	15
Section 4.5: accuracy-energy Pareto frontier	("Pareto front" OR "Pareto frontier") AND ("quantization" OR "pruning") AND ("measured energy" OR "energy consumption") AND "accuracy"	639	15

Screening used the fifteen highest-ranked records per query. Records returned is the index’s reported total for the query.

Two limits follow. The supplementary search used one index and screened only the fifteen highest-ranked records per query, so it lowers rather than removes the risk that a relevant study was missed; and a purposive corpus can over-represent the questions its assembler already found interesting. Every statement in this

review of the form “no study” is therefore scoped to the corpus and the supplementary search, and should be read as “no study located by the procedure described in this section.”

2 Energy Measurement Methodology

Energy claims about machine learning rest on measurements, and the measurement methods themselves differ in what they capture, how accurate they are, and how much they cost to run. This section examines the three layers of the measurement stack. Software estimators derive energy from runtime behavior and hardware specifications. Hardware performance counters read power from the processor itself. External meters provide the reference against which the other two are judged. A fourth subsection collects the methodological decisions, idle baselines, warm-up periods, batch sizes, and reporting conventions that determine whether two measurements can be compared at all.

2.1 Software-Based Estimators

Energy measurement in machine learning began with proxies: algorithms that infer power from runtime and hardware properties rather than observing it directly. Two families exist. Static estimators multiply a workload's duration by a hardware assumption such as thermal design power. Dynamic estimators sample hardware counters during execution and integrate the observed power over time. The ML Emissions Calculator represents the static family, while CodeCarbon, pyRAPL, Scaphandre, Carbontracker, and Eco2AI belong to the dynamic family, alongside the framework-level libraries PyJoules and MELODI (Husom et al., 2025). CodeCarbon samples GPU power through NVML, so a software tool and a hardware counter can share the same underlying signal (Caravaca et al., 2025).

The estimation method determines what a tool can be trusted for. Fischer ground-truthed both families against external hardware meters on a fixed platform. The static calculator produced errors from roughly minus 40 percent to plus 40 percent, with the error ranging from minus 124 W to plus 195 W across the models tested, while CodeCarbon consistently underestimated power draw by 15 to 25 percent and by 46 to 90 W (Fischer, 2026). The dynamic tool's underestimate was systematic, and bias of that kind does not cancel across measurements the way random error does.

Tool builders increasingly train statistical models on measured data instead of relying on design-power constants. Caravaca and colleagues measured more than 32,500 inference runs across 21 GPU configurations and 155 language models, then fitted regression models that predict energy from architecture, hardware, and usage features. Their generalist model reached a mean absolute percentage error of about 8 percent on held-out data and 10.5 percent on an unseen model family (Caravaca et al., 2025). The ground truth for that training was the authors' own NVML sampling, so a learned estimator inherits whatever bias the underlying counter carries.

Benchmarking efforts push in a different direction by standardizing what gets measured. MLPerf Power defines a methodology spanning microwatts to megawatts, using external micro-power instruments for microcontroller workloads, certified power meters for edge devices, and node-level counter telemetry for datacenters, and it rejects thermal-design-power and power-supply-unit figures as inaccurate proxies (Tschand et al., 2024). The suite reports 1,841 reproducible measurements from 60 systems, with small workloads near 5.64 mW and training runs reaching 500 kW. Accuracy documentation is mandatory rather than optional, and measurement is restricted to the execution phase, which excludes data preparation.

A recurring pattern across these tools is that accuracy is asserted without independent verification. Fine-grained instrumentation work observed that existing tools under-report energy and carbon, and that sampling intervals exceeding ten seconds are too coarse for attributing energy to individual API calls (Rajput et al., 2023). CodeCarbon carried more than 1,800 GitHub stars and thousands of daily downloads at the time of the ground-truthing study, which placed it alongside documented errors of 15 to 25 percent (Fischer, 2026). Wide adoption and measurement bias are compatible, and adoption does not establish accuracy.

A parallel body of work profiles inference energy at the level of the software stack rather than the model. Fadel Argerich and Patiño-Martínez built a software tool that separates CPU, memory, GPU, and storage energy on Linux and used it to study how model size, architecture, batch size, and weight quantization affect large language model inference on a workstation with an Intel Core i9-9900K and an NVIDIA RTX 4090; for the accuracy of the underlying RAPL counters they rely on prior work reporting errors below 2 percent on Haswell-generation processors rather than validating the counters against a meter themselves (Fadel

Argerich & Patiño-Martínez, 2024). Rajput and Sharma’s CodeGreen platform addresses the granularity problem: it decouples instrumentation from measurement so that Intel RAPL, NVIDIA NVML, and AMD ROCm can be polled independently at tunable checkpoints, and validation against the Computer Language Benchmarks Game reached an R^2 of 0.9934 against RAPL ground truth (Rajput & Sharma, 2026). The same validation exposes an overhead-granularity trade-off: after normalization, coarse instrumentation added 9.1 to 9.6 percent overhead for C and C++ programs, while an intentional stress test with one million checkpoints in the nbody benchmark reached 607 percent (Rajput & Sharma, 2026). Aquino-Brítez and colleagues took the comparative route, training six architectures on two NVIDIA GPUs and comparing CodeCarbon and CarbonTracker against a sensor-based OpenZmeter reference: CodeCarbon tended to underestimate energy relative to the hardware meter, with statistically significant differences, and CarbonTracker reported substantially higher values overall (Aquino-Brítez et al., 2025). Chung and colleagues’ ML.ENERGY Benchmark pursues standardization from the deployment side, measuring 40 model architectures across six tasks under realistic serving conditions and reporting that automated configuration recommendations can yield energy savings exceeding 40 percent without changing what the model computes (Chung et al., 2025). Each of these tools addresses a different part of the same problem, and none has been validated on compressed models (Gap 1, Section 7.2).

Layer-wise estimation offers a fourth route between whole-program proxies and external metering. Sardi and colleagues’ WattLayer decomposes inference energy into per-layer contributions, using a dataset of 99,411 samples from 164 image, text, and audio models drawn from a measurement campaign of over 100,000 energy measurements covering up to 295 architectures; its reference readings come from CodeCarbon, which combines RAPL for the CPU with nvidia-smi for the GPU (Sardi et al., 2026). On 93 image-classification models held out from training it reaches a mean absolute percentage error of 40.1 percent, against 63.9 and 99.8 percent for two variants of one baseline family and 132.6 and 833.7 percent for two variants of another (Sardi et al., 2026). A naïve summation of layer-level estimates underestimates the measured total by approximately 25 percent, which the authors attribute to memory management, data movement, and frequency-setting overhead that isolated layer profiling cannot see, and they calibrate a correction factor of 1.25 to close the gap (Sardi et al., 2026). The correction factor is empirical rather than derived, which is itself evidence for this review’s argument: a model’s energy is not the sum of the energy its parts consume in isolation.

The absolute error matters as much as the comparison. An estimator that outperforms its baselines at a 40.1 percent mean error still misses by more than the compression effects this review examines: Chang and colleagues measured pruning-induced energy changes between -11.8 and $+17.2$ percent across INT8 delegates at identical FLOPs (Chang et al., 2026), Chung and colleagues found FP8 changing energy by between -29 and $+56$ percent relative to BF16 depending on batch size (Chung et al., 2026), and Paula and colleagues report a 6.7 percent energy increase when DistilBERT is pruned (Paula et al., 2025). The best available layer-wise estimator in the corpus therefore cannot yet resolve the effects it would be asked to predict, and the reference it is trained against is itself a software counter rather than an external meter.

2.2 Hardware Performance Counters

Hardware counters read power where it is consumed. Intel introduced RAPL with Sandy Bridge in 2011, and the interface exposes four domains: the package, the cores (PP0), the uncore or graphics domain (PP1), and DRAM (Raffin & Trystram, 2024). The domains nest only partially. Core and uncore sit inside the package, while DRAM does not, and on AMD Zen processors only the package and core domains are available. Accuracy depends on generation as well: Haswell-era parts track power closely, and the interval between generation and reliability is not documented for every part in use.

The access mechanism shapes both overhead and privilege requirements. A comparative study re-implemented four access paths, MSR, powercap, userspace perf-events, and eBPF, and measured their read latencies, finding perf-events fastest and powercap slowest (Raffin & Trystram, 2024). On a loaded CPU, the overhead stayed negligible even at 1,000 samples per second, while the idle power penalty differed sharply by mechanism and reached tens of watts for powercap and MSR on an Intel server. Privileges are non-negotiable: MSR access requires raw I/O capability, perf-events requires a performance-monitoring capability, and eBPF requires elevated BPF privileges.

The act of measurement changes the system, a phenomenon the literature calls the observer effect. Diamond and Stoico measured the time overhead of seven RAPL-based tools across six benchmarks and found a range

from 0.25 percent to 46.75 percent at the interface's maximum sampling rate of 1 kHz (Diamond & Stoico, 2026). CodeCarbon imposed the worst case at 46.75 percent on one benchmark, where its energy overhead exceeded 40 percent, and Scaphandre added 17.6 percent on another. A single system-call read can consume upwards of 0.01 ms, so restricting a study to the core domain reduces interference when memory and graphics demand little attention.

GPU and edge counters follow different rules. NVML reports NVIDIA GPU power, is limited to NVIDIA hardware, and cannot separate CPU from GPU consumption within a running process (Husom et al., 2025). On Jetson boards, the built-in INA3221 sensors underestimated true power by up to 50 percent before calibration; after fitting per-device linear models against an oscilloscope reference, errors fell to within 0.8 to 3 percent (Shalavi et al., 2023). Boot-time current spikes shorter than a millisecond appeared only to the external instrument, which indicates that internal sensors miss transient behavior.

On mobile devices, the power management integrated circuit offers component-level attribution without external equipment. A profiling framework built on the PMIC rails of a Pixel 9 disaggregated CPU, GPU, TPU, and DRAM energy, reading a power accumulator every 16 ms from a 125 Hz analog-to-digital converter (Chang et al., 2026). Against a direct-integration reference on the same trace, it achieved a mean absolute percentage error between 4.33 and 8.14 percent, with the profiler itself adding 2.72 percent overhead. That validation compared the framework against an internal reference rather than an independent meter, which leaves its absolute accuracy an open question (Gap 4, Section 7.2).

2.3 Ground Truth Validation

External meters provide the reference against which software tools are judged, and the instruments in use differ widely in cost and resolution. Joulescope samples current at 2 MHz and logs accumulated energy, and it was used to profile 28 quantized language models on a Raspberry Pi 4 (Husom et al., 2025). The same study reached the meter's three-ampere continuous limit on a Raspberry Pi 5 and fell back to a lower-resolution USB meter, which shows that instrument choice is constrained by the target platform.

Fischer validated estimation tools against a smart socket and a digital energy meter, reading the meter display through a camera and optical character recognition because the device exposed no digital interface (Fischer, 2026). Bartoli and colleagues measured microcontroller energy directly, placing a 50 milliohm shunt resistor in series with the core and capturing the voltage across it with a Tektronix oscilloscope, then isolating inference from surrounding phases with a dual-GPIO trigger (Bartoli et al., 2025). Sierra-Herrera and colleagues measured system power with a P4400 consumer power meter rated to 0.2 to 2 percent accuracy, and reported no validation of any software tool against that meter (Sierra-Herrera et al., 2026).

The validation protocol is conceptually simple: run the same workload, compare the tool output with the meter reading, and report the difference. Direct validation of CodeCarbon against an external meter appears in few corpus papers, and the ground-truthing study that performed it is framed by its own author as a first step limited to a single hardware platform (Fischer, 2026). External references range from consumer energy meters and smart sockets costing less than 30 euros (Fischer, 2026) to bench analyzers and oscilloscopes, and the more capable instruments occupy a laboratory bench and impose sample-rate limits of their own, which helps explain why they appear in dedicated measurement studies rather than in routine reporting.

The reference instruments carry uncertainty of their own, and any validation inherits it. Fischer notes that external measurement hardware can itself be inaccurate, without quantifying that error for the smart socket and consumer meter used as ground truth (Fischer, 2026). Sierra-Herrera and colleagues' consumer meter is specified at an accuracy between 0.2 and 2 percent (Sierra-Herrera et al., 2026), an order of magnitude tighter than the estimator errors in Section 2.1 but not negligible against single-digit compression effects. MLPerf Power requires a SPEC-certified analyzer for inference submissions and reports that such meters show larger error bars on very low-power devices, because power adapters below 75 W produce a high crest factor, and that full-system readings on small devices include components such as wireless transceivers and idle interfaces that do no inference work (Tschand et al., 2024). Sampling rate and bandwidth matter at the timescale of a single inference: Shalavi and colleagues' reference oscilloscope sampled at 100 million samples per second with a 10 MHz bandwidth and captured boot-time current peaks shorter than a millisecond (Shalavi et al., 2023), and the Joulescope case described above shows that the reference instrument itself can be forced to change with the device under test (Husom et al., 2025). Ground truth therefore means a reference whose accuracy class, sampling rate, and measurement boundary are stated, not

an instrument whose reading is exact, and a tool validation that does not report these properties of its reference cannot bound the error it attributes to the tool.

Table 2. Measurement tools and methods in the corpus.

Tool or method	Mechanism	Reported error, accuracy, or overhead	Reference used	Platform support	Source
ML Emissions Calculator	Static (TDP × time)	−40% to +40%; −124 W to +195 W	Smart socket and consumer energy meter	Any (assumed TDP)	Fischer (2026)
CodeCarbon	Dynamic (RAPL, NVML)	Underestimated power by 15–25% (46–90 W); underestimated energy vs. OpenZmeter; time overhead up to 46.75% at 1 kHz	Smart socket and energy meter; OpenZmeter; uninstrumented runtime	Intel/AMD CPUs, NVIDIA GPUs	Fischer (2026); Aquino-Brítez et al. (2025); Diamond & Stoico (2026)
CarbonTracker	Dynamic	Substantially higher energy values than hardware meter	OpenZmeter	NVIDIA GPUs (as tested)	Aquino-Brítez et al. (2025)
Scaphandre	Dynamic (RAPL)	Time overhead up to 17.6% at 1 kHz	Uninstrumented runtime	RAPL-capable CPUs	Diamond & Stoico (2026)
Intel RAPL	On-chip counter	<2% on Haswell (cited from prior work, not re-tested)	Prior external validation	Intel (package, core, uncore, DRAM); AMD Zen (package, core only)	Fadel Argerich & Patiño-Martínez (2024); Raffin & Trystram (2024)
nvidia-smi / NVML	On-board counter	≈5% (cited from prior work, not re-tested)	Prior study	NVIDIA GPUs only	Sardi et al. (2026)
Jetson onboard power sensors	On-board counter	Underestimate up to 50%; ±3% after per-device calibration	PicoScope 2204A oscilloscope (100 MS/s)	NVIDIA Jetson	Shalavi et al. (2023)
Pixel 9 PMIC rail profiler	PMIC accumulator (125 Hz, read every 16 ms)	MAPE 4.33–8.14%; profiling overhead 2.72%	Direct integration of the same trace (internal)	Google Pixel 9	Chang et al. (2026)
CodeGreen	Checkpointed polling (RAPL, NVML, ROCm)	$R^2 = 0.9934$ vs. RAPL; overhead 9.1–9.6% (C/C++, coarse)	RAPL	Intel/AMD CPUs; NVIDIA/AMD GPUs	Rajput & Sharma (2026)
WattLayer	Learned layer-wise model	MAPE 40.1% (93 held-out image models)	CodeCarbon (RAPL, nvidia-smi)	NVIDIA GPUs	Sardi et al. (2026)
Random-forest energy model	Learned model	MAPE 8.03% (cross-validation)	CodeCarbon (NVML)	NVIDIA GPUs (21 configurations)	Caravaca et al. (2025)
Joulescope JS110	External DC meter	2 MHz sampling; ±3 A continuous limit	—	DC-powered edge devices	Husom et al. (2025)
P4400 consumer meter	External AC meter	Accuracy 0.2–2% (specified)	—	AC-powered systems	Sierra-Herrera et al. (2026)
SPEC-certified analyzer (e.g., Yokogawa WT310)	External AC analyzer	Larger error bars below 75 W (crest factor)	—	AC-powered systems	Tschand et al. (2024)
Shunt + oscilloscope; GPIO-synchronized DMM	External, phase-triggered	50 mΩ shunt, ±1% tolerance (Tektronix MSO64B); Keysight 34465A DMM	—	Microcontrollers	Bartoli et al. (2025); Fanariotis et al. (2025)

Errors are as reported by the cited source under that source’s conditions; figures marked as cited are taken by the source from prior work rather than measured by it.

2.4 Measurement Challenges and the Standardization Gap

Measurement must decide what counts as a workload’s energy. Husom and colleagues measured idle power separately at 2.85 W, with a standard deviation of 0.17 W over 104 minutes, and subtracted it from inference energy (Husom et al., 2025). Rajput’s fine-grained method waited for the platform to reach a stable baseline before each call and subtracted it as well (Rajput et al., 2023). Bartoli’s TinyML method treated idle as a defined phase, excluded it from measurement, and listed baseline subtraction as future work (Bartoli et al., 2025), while other studies report total system energy without subtraction. A reader comparing net and gross figures across papers is comparing different quantities under the same name. The difference is concrete at the scale of a single inference: at the 2.85 W idle power Husom and colleagues measured, a 50 ms inference window carries about 0.14 J of idle energy, which a gross figure includes and a net figure does not, before any task energy is counted.

Fine-grained attribution has a counterpart outside single-machine benchmarking. Sharma and Fuerst built a marginal-energy-based validation methodology for serverless functions and report per-invocation energy footprints accurate to within 99 percent of the marginal-energy ground truth across FaaS workloads on three hardware platforms (Sharma & Fuerst, 2024). Their accounting also allocates shared and embodied emissions, which they note prior estimates place at up to 20 percent of lifetime emissions for servers and more than 80 percent for mobile phones. The idle-versus-task boundary described in this section is one instance of that more general accounting problem: deciding what a unit of computation is responsible for.

First runs behave differently from steady state. Sierra-Herrera and colleagues observed a higher cost in the first training epoch that diminished thereafter, and treated later epochs as steady state (Sierra-Herrera et al., 2026). MLPerf Power uses an initial run to set instrument ranges before fixed-range measurement begins, and loops workloads shorter than 60 seconds so that power is averaged over valid samples (Tschand et al., 2024). Protocols that discard the first run and protocols that keep it produce different numbers for the same workload.

Batch size changes hardware utilization, so energy per batch and energy per sample diverge as batch size grows. One energy model restricted itself to batches above five prompts to avoid overfitting to small-batch behavior (Caravaca et al., 2025). Reporting a single energy figure without stating the batch size conceals the variable that most affects utilization on accelerators.

Reporting conventions differ as well. Husom and colleagues reported mean and standard deviation throughout and noted variance in joules per token that sometimes exceeded the mean, which signals that single-run figures are unreliable at the device level (Husom et al., 2025). Bartoli and colleagues averaged 1,000 acquisitions per model (Bartoli et al., 2025), and Sierra-Herrera and colleagues reported mean and standard deviation over five runs without confidence intervals (Sierra-Herrera et al., 2026). The absence of a shared minimum for sample size and dispersion makes cross-study comparison depend on what each author chose to publish. Across the 21 primary empirical studies summarized in Table 3, the batch size used for the energy measurement is stated in 10, an idle baseline is subtracted in 4, and dispersion across repeated measurements is reported in 9; the remaining studies do not report these items in text that a full-text search could locate.

Table 3. Characteristics of the primary empirical studies in the corpus.

Study	Hardware	Models	Compression states	Instrument	Idle baseline	Batch size stated	Dispersion	Phase
Alizadeh & Castor (2024)	Laptop: Intel Core i7-11850H, RTX 3070	ResNet, MobileNet, BERT-base	None (runtime comparison)	pyRAPL (CPU); nvidia-smi, 10 ms (GPU)	Subtracted	Y	N	Inference
Bartoli et al. (2025)	STM32N6570-DK	TinyML models	None	50 mΩ shunt, Tektronix MSO64B	Separate phase, excluded	N	Y	Inference
Caravaca et al. (2025)	21 GPU configurations (cloud)	155 LLMs	INT4 (AWQ, GPTQ) and FP8 variants	CodeCarbon (NVML)	N	Y	N	Inference
Chang et al. (2026)	Google Pixel 9 (CPU, GPU, TPU)	ResNet50V2	FP32/INT8; five pruning variants	PMIC rails	N	N	Y	Inference
Chung et al. (2026)	NVIDIA H100, B200	46 generative models	BF16 vs. native FP8	Zeus	N	Y	N	Inference
Dos Santos Ferreira et al. (2026)	Five Intel/AMD machines, three with GPUs	CNNs and language models	None (hyperparameter search)	CodeCarbon	N	Y	Y	Training and inference
Fadel Argerich & Patiño-Martínez (2024)	Intel Core i9-9900K, RTX 4090	Multiple LLMs	Weight quantization	Own tool (CPU, memory, GPU, storage)	N	Y	N	Inference
Fanariotis et al. (2025)	Alif Ensemble E7 (MCU + NPU)	Six models	None (CPU vs. NPU)	Keysight 34465A DMM, GPIO-triggered	Subtracted	N	N	Inference
Fischer (2026)	Single CPU/GPU platform	Vision and language models	None	Smart socket, energy meter vs. CodeCarbon	N	Y	Y	Inference
Husom et al. (2025)	Raspberry Pi 4 (Pi 5 supplementary)	28 Ollama LLMs	Weight-only PTQ levels	Joulescope JS110; USB meter on Pi 5	Subtracted	N	Y	Inference
Lahmer et al. (2022)	Jetson TX2, Xavier NX	Synthetic conv/FC layers	None	Onboard sensors, 0.1 ms slots	N	N	N	Inference (CPU)
Mei & Stamp (2025)	Kaggle cloud	Nine pre-trained models	Pruning, low-rank, capacity reduction	Software estimate (CPU, GPU, RAM)	N	N	N	Training
Paula et al. (2025)	NVIDIA T4 (listed)	BERT, DistilBERT, ALBERT, ELECTRA	Pruning, distillation, PTQ, combinations	CodeCarbon	N	Y (fine-tuning)	N	Not separated
Rajput & Sharma (2024)	Not located	One LLM	Five 4-bit PTQ methods	CodeCarbon	N	Y	N	Inference
Sierra-Herrera et al. (2026)	x86 desktop and laptop, Apple M4, Raspberry Pi 5, cloud	CNN, RNN, LSTM, XGBoost	None	P4400 wall meter; nvidia-smi	N	Y	Y	Training
Tu et al. (2023)	Smartphones (e.g., Huawei P40 Lite)	MobileNetV2 and other DNNs	FP32 vs. INT8	Monsoon power monitor	N	N	N	Inference
Tummalapalli et al. (2026)	Raspberry Pi 5 + Hailo-10H, Galaxy	One 4-bit LLM	Q4_0	INA219 at 1 kHz; nvidia-smi; fuel	Reported, not	N	Y	Inference

	S24 Ultra, iPhone 16 Pro, RTX 4050 laptop			gauge	subtracted			
Xiang (2026)	Raspberry Pi 4	ImageNet, CIFAR-10, Speech Commands models	Structured compression, distillation, mixed precision	FNIRSI FNB48 USB meter (100 Hz, calibrated)	N	Y	Y	Inference
Xu et al. (2023)	Four Jetson Nano	Vision transformers	Decomposition with distillation	Monsoon HVPM	N	N	N	Inference
Zou et al. (2026)	Phone, laptop, development board	Quantized LLMs	4-bit	In-house ADC power monitor	Subtracted	N	N	Inference
Giedra et al. (2026)	Jetson Orin Nano Super	Twelve LLMs (<1B to 8B)	Quantized GGUF deployments	tegrastats + external DMM	Recorded, not subtracted	N	Y	Inference

N = not located in the study's full text. Batch size stated refers to the energy measurement itself. Dispersion = standard deviation or confidence interval across repeated energy measurements.

The tools exist in abundance, but the protocol that would make their outputs comparable does not yet command agreement. MLPerf Power identified eleven requirements for a measurement methodology and positioned itself as the only method meeting all of them, defining uniform logging, reporting templates, sampling criteria, and mandatory disclosure of estimation methods (Tschand et al., 2024). A systematic review of green machine learning reached a compatible conclusion from the survey side, naming the absence of standardized reporting metrics and the difficulty of reproducing energy results as open problems (Santos et al., 2026). TinyML benchmarking work argued that MLPerf Tiny attributes non-inference phases to inference and reports results without accounting for measurement uncertainty (Bartoli et al., 2025). Each paper that reports energy without a shared protocol adds one more data point that the next review cannot compare to the last.

3 The FLOPs-Energy Disconnect

Energy-aware model design has long leaned on a compact rule: a model that performs fewer floating-point operations costs less to run. The rule survives because it is convenient. FLOPs follow from the architecture alone, independent of the hardware, the software stack, and the batch size, and they reduce a design to a single comparable number (Schwartz et al., 2019). Measurement studies complicate the rule at every level. This section gathers the evidence that computational load and energy diverge, and identifies the mechanisms, memory traffic and batch size, that displace the proxy.

3.1 The FLOPs Myth

The assumption that energy tracks computation has an intuitive form: energy equals the number of operations divided by the energy cost of each operation, and both terms are treated as roughly constant. Desislavov and colleagues tested the assumption against fourteen years of inference data and found that the second term moves faster than the first. They estimated per-inference energy as model FLOPs divided by the floating-point throughput per watt of the GPU current at the model's release date, then compared the resulting energy trend with the raw compute trend (Desislavov et al., 2021). The exponential growth in FLOPs softened once hardware efficiency was divided out, and remained exponential only for the frontier of highest-accuracy models, where the general-use fit ran nearly flat.

The proxy also fails at the level of the architecture itself. Parameter count and compute diverge because the same parameter budget distributes differently across depth, width, and input resolution, each of which changes the operation count non-linearly. Desislavov and colleagues measured the correlation between parameter count and FLOPs at 0.772 for convolutional models and 0.994 for transformers, so a parameter count that predicts compute well within one architecture family predicts it poorly across families (Desislavov et al., 2021). Their reading of the wider literature adds a condition that will recur throughout this review: energy and FLOPs correlate closely for a single architecture, and the correlation weakens as architectures are mixed.

Direct measurement confirms the weakening. Dos Santos Ferreira and colleagues tuned energy efficiency and accuracy jointly across a convolutional architecture search and computed the correlation between common proxy metrics and measured inference energy. Parameter count correlated with energy at a Pearson coefficient of minus 0.12, and FLOPs at 0.26, both far from the near-unit correlation the proxy assumes (Dos Santos Ferreira et al., 2026). A separate prediction study on edge devices compared a FLOPs-based energy predictor with a kernel-level predictor and reported accuracies of 31.3 percent and 86.2 percent, respectively, so the proxy discarded a large fraction of the signal that a finer model recovered (Tu et al., 2023).

The clearest demonstrations place two models side by side. Xu and colleagues compared a Twins-SVT-S model at 2.9 GFLOPs consuming 661.71 mJ with a decomposed variant at 5.9 GFLOPs consuming 221.08 mJ, a case where the model with twice the nominal compute used a third of the energy (Xu et al., 2023). Lahmer and colleagues, modeling energy on Jetson boards, found that energy grew linearly with the multiply-accumulate count when the input size, kernel size, or stride changed, and non-linearly once the number of output feature maps was varied, with the marginal energy of an operation falling as the feature maps multiplied (Lahmer et al., 2022). Their conclusion states the problem for the proxy directly: the pure number of operations is not sufficient to characterize the energy consumption of the boards. Each of these results points away from the operation count and toward how the hardware moves and reuses data.

Chung and colleagues' large-scale measurement study reinforces the point at a scale the smaller comparisons cannot reach: across 46 models, 7 tasks, and 1,858 configurations on NVIDIA H100 and B200 GPUs, task type alone produces up to 25-fold energy differences, video generation can consume more than 100 times the energy of image generation, and GPU utilization differences between otherwise similar workloads produce 3- to 5-fold energy differences (Chung et al., 2026). None of these multipliers is visible to a FLOPs count, because FLOPs describes the algorithm and these differences arise from how the algorithm meets the hardware (Gap 2, Section 7.2).

3.2 Memory Bandwidth as the True Bottleneck

The roofline model explains why the operation count misleads. A workload's position depends on its arithmetic intensity, the ratio of computation to bytes moved, against the machine's balance of peak compute and memory bandwidth. Workloads with high arithmetic intensity are compute-bound, and their runtime follows the operation count. Workloads with low intensity are memory-bound, and their runtime follows data movement, which the operation count does not describe. Much inference sits on the memory-bound side of the roof, particularly at small batch sizes and in the layers that move large activations and weights.

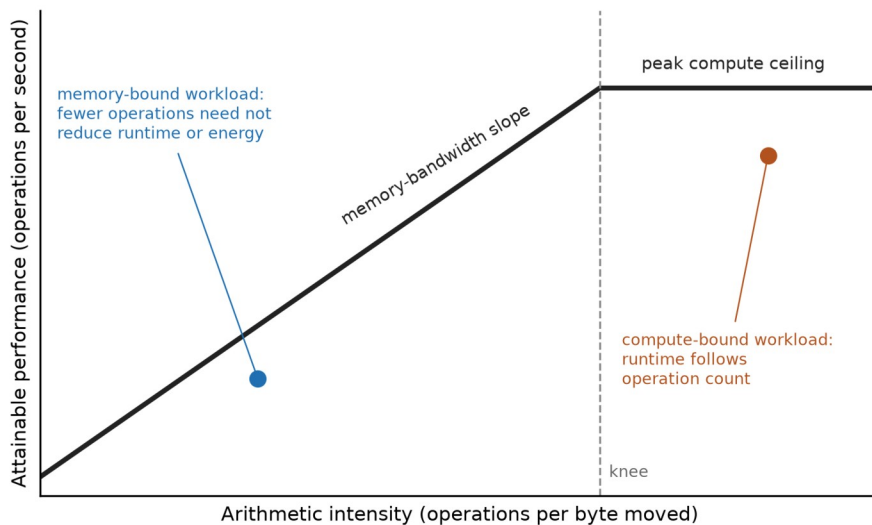


Figure 1. Schematic of the roofline model (conceptual, not drawn from measured data). Workloads left of the knee are limited by memory bandwidth, so reducing their operation count need not reduce runtime or energy; workloads right of the knee are limited by peak compute.

Li and colleagues measured this balance for vision transformers on mobile devices and found that transformers generally have lower arithmetic intensity than convolutional networks, which gives them a stronger tendency to memory-bound behavior (Li, Paolieri & Golubchik, 2025). They tested the difference by scaling two resources independently. Scaling memory frequency sped up 75 percent of the transformers by more than 3.40 times while speeding up only 28 percent of the convolutional networks, and scaling the CPU clock from 0.83 to 2.84 GHz sped up 89 percent of transformers by less than 1.93 times while moving only 12 percent of the convolutional networks. A memory-bound workload gains from faster memory and barely reacts to faster compute, and the measurements separate the two families along exactly that axis.

Their latency results show the same effect without any energy instrumentation. A transformer using 5.00 GFLOPs incurred 1.75 times the latency of a convolutional network using 4.95 GFLOPs, with near-identical

operation counts producing very different runtimes (Li, Paolieri & Golubchik, 2025). The layer that consumed the time was a GELU activation whose latency varied by 2.85 times depending on the input value, a data-dependent cost that no static FLOPs count can express.

Where the energy concentrates reinforces the memory-side explanation. Tu and colleagues decomposed model energy by layer on edge devices and found that convolution with batch normalization and activation accounted for 93.97 percent of CPU model energy and 87.74 percent on the GPU, with the cost dominated by the movement and processing of feature maps rather than by arithmetic (Tu et al., 2023). They also observed that intra-kernel power rises to a peak within roughly ten milliseconds and then flattens, with ramp-up times of 10.5 ms on a Kirin 990, 10.2 ms on a Kirin 810, and 6.2 ms on a Snapdragon 855. Short kernels spend a large share of their energy in the ramp, a fixed cost that scales with the number of launches rather than with the operation count.

Transformers on mobile devices also expose how software decides whether a precision reduction becomes a speedup. Li and colleagues found that quantized linear operations in PyTorch Mobile ran slower than their floating-point counterparts on the large and medium cores, because the integer kernels of the QNNPACK backend were slower than the floating-point NEON kernels of XNNPACK, and acceleration appeared only on the small cores (Li, Paolieri & Golubchik, 2025). A memory-bound layer plus a slower kernel is a case where lowering precision raises both latency and energy, and the mechanism lies in the software stack rather than in the arithmetic.

3.3 Batch Size as a Hidden Variable

Batch size determines how well the hardware is utilized, so it governs a workload's position on the roof and, through that, its energy per unit of work. Dos Santos Ferreira and colleagues quantified the effect over a hyperparameter search and found an optimal batch size of 831 for one model, yielding 2.67 tokens per joule, a thirty-eight-fold efficiency gain over batch size one and a reduction of 13.95 joules per token (Dos Santos Ferreira et al., 2026). The same measurement showed that small models and small batches leave GPU parallelism underused, which inflates the energy of every unit of work they complete.

The consequence for compression is direct. A compression technique that lowers the operation count changes how a workload fills the machine, and its energy benefit depends on whether the resulting workload still saturates compute. At small batches, where utilization is already low, removing operations can leave energy nearly unchanged or move it the wrong way. Caravaca and colleagues restricted their LLM energy model to batches above five prompts precisely to avoid fitting small-batch behavior in which the utilization regime shifts (Caravaca et al., 2025), and the energy-per-token figures reported by Husom and colleagues on a Raspberry Pi 4 ranged from 17.60 joules per token for a one-billion-parameter model in fp16 to 3.75 joules per token for its most efficient quantized variant, a spread that reflects both the precision change and the token-generation dynamics the batch modulates (Husom et al., 2025).

Reporting conventions hide the variable. A single energy figure quoted without its batch size describes one point on a steep curve, and two studies can disagree about whether compression saves energy while measuring the same model if they differ on this axis. Alizadeh and Castor found that runtime infrastructure and batch configuration changed both the absolute energy and the relative ranking of models across PyTorch, TensorFlow, MXNet, and ONNX (Alizadeh & Castor, 2024), which means the batch size is part of the result and not a footnote to it.

Chung and colleagues isolated the mechanism with a controlled precision comparison. Running Qwen3 235B A22B on eight H100 GPUs in both BF16 and FP8, they found FP8 losing on both energy and latency at batch sizes 8 to 16, consuming 13 to 56 percent more energy (median 30 percent more) and up to 26 percent higher latency, because at low batch size the extra overhead of quantization and de-quantization outweighs any throughput advantage while the GPU sits underutilized regardless of precision. At batch sizes 65 to 256, the ranking reverses, and FP8 wins on energy by up to 29 percent (median 11 percent less) (Chung et al., 2026). A single quantization method therefore does not have a fixed sign for its energy effect; it has a batch-size-dependent crossover, and a report that quotes one number without the batch size at which it was measured has reported half of a result.

3.4 Latency and Energy Are Not Equivalent

Because energy is power integrated over time, and latency is time, a fast configuration can spend more energy than a slow one when its power draw is high enough. The clearest measurements come from

accelerators at small workloads. Tu and colleagues found that running a convolution on a mobile GPU consumed more than 6.6 times the energy of the same operation on the CPU at small feature sizes, even though the GPU finished sooner, while at the model level the GPU saved between 49 and 79 percent of the energy relative to the CPU (Tu et al., 2023). The sign of the GPU's energy advantage flipped with workload size, and latency tracked energy in neither direction reliably.

Compression experiments reproduce the divergence. Xu and colleagues reported that their decomposed transformer on a Jetson Nano cut latency by 2.16 times and energy by 61.83 percent relative to a baseline, and by 66.61 percent relative to MobileViT-S, so the energy gain exceeded the latency gain (Xu et al., 2023). Bartoli and colleagues found the opposite asymmetry in a TinyML setting: lowering the microcontroller voltage left latency nearly unchanged while reducing total energy from 277.1 to 197.6 microjoules on one model and cutting the energy-delay product by an average of 25.5 percent across the benchmark suite (Bartoli et al., 2025). A method can move energy substantially while leaving time flat.

The countervailing evidence is that latency and energy often track well in practice. Alizadeh and Castor measured a Spearman correlation of 0.99 between inference time and energy across their runtime configurations (Alizadeh & Castor, 2024), and Sander and colleagues argued that parameter count and FLOPs correlate weakly with latency on modern hardware while treating latency, memory, and energy as distinct objectives in a search (Sander et al., 2025). The two findings coexist because latency is a reasonable proxy on a fixed platform and a poor one across platforms and workloads, where the power envelope and the utilization regime change. A latency leaderboard answers how quickly a model runs and leaves open how much energy the answer costs, which is the question a green deployment decision actually asks.

4 Model Compression and Energy Trade-offs

Compression techniques shrink a model's stored size or its computational footprint, and each is promoted on a version of the same promise: less of the model means less energy. The preceding section showed that the promise fails as a general rule. This section examines the three main families, quantization, pruning, and knowledge distillation, and the combinations that production systems use in practice, to establish where compression does reduce energy and where it does not.

4.1 Quantization

Quantization reduces the number of bits used to store weights and activations. Nagel and colleagues define the two dominant regimes. Post-training quantization (PTQ) takes a trained network and quantizes it with little or no data, without end-to-end training, and is described as a push-button approach. Quantization-aware training (QAT) instead retrains the network with quantization simulated in the training pipeline, so the parameters adapt to the reduced precision (Nagel et al., 2021). The same white paper states the hardware rationale that motivates the field: moving from 32 to 8 bits reduces the memory cost of storing tensors by a factor of four and the cost of matrix multiplication by a factor of sixteen, because the energy of digital arithmetic scales between linearly and quadratically with bit width. On that arithmetic basis, an INT8 model should cost far less energy than its FP32 original.

The prediction holds at the level of the arithmetic and not at the level of the system. The white paper measures no energy, latency, or power; it reports accuracy under simulated quantization on floating-point hardware and states the efficiency case analytically (Nagel et al., 2021). A survey of edge deployment compiles the realized effects and finds INT8 delivering a four-fold model-size reduction for a one to three percent accuracy cost, sub-four-bit formats reaching eight to sixteen times reduction, and binarization reaching sixteen to thirty-two times, with the caveat that the lower bit widths require QAT or distillation to retain accuracy (Cordova-Cardenas et al., 2025). The same survey reports a hardware-support gap: non-uniform quantization is not natively supported by all inference engines, and adoption is bounded by bit-width support, memory-bandwidth limits, and heterogeneous toolchains. A reduction the backend cannot execute does not reduce energy.

The distinction between weight-only and weight-activation quantization sharpens the point. Li and colleagues evaluate quantized large language models and separate the two cases by what the hardware can accelerate. Weight-only quantization speeds up the memory-bound matrix-vector products of decoding, and weight-activation quantization enables low-precision tensor cores for the compute-bound matrix multiplications of prefill. Quantizing activations alone is the case that resists acceleration: in prefill, the weights stay in a wider format so the low-precision units cannot be used, and in decoding at small batch

sizes, the bottleneck is weight access, so reducing activation memory yields no notable speedup (Li et al., 2024). The same evaluation records the accuracy cliff, with four-bit weight-activation quantization causing a complete loss of performance for most models and two-bit formats a significant loss, while four-bit weight quantization held losses under two percent (Li et al., 2024).

A recurring obstacle is the cost of moving between formats. Low-bitwidth data is de-quantized to a wider type such as FP16 before computation (Li et al., 2024), and on mobile CPUs the integer kernels of the QNNPACK backend ran slower than the floating-point Neon kernels of XNNPACK for quantized linear operations (Li, Paolieri & Golubchik, 2025). The arithmetic shrinks while the data movement and the conversion remain, which is the mechanism behind a divergence the literature notes repeatedly: nominal compression, the bit-width reduction the model reports, differs from realized compression, the work the hardware actually performs. Each of these mechanisms is a specific way that the INT8-equals-one-quarter-energy expectation gives way to something closer to the memory-bound behavior described in Section 3.

4.2 Pruning

Pruning removes weights or structures from a trained model. The distinction that governs its energy behavior is whether the removal targets individual weights or whole units. Unstructured pruning zeros individual weights throughout the tensor, and structured pruning removes entire filters or channels so that the resulting tensor dimensions shrink.

Unstructured pruning leaves the arithmetic intact. Francy and Singh report that pruning individual weights lowers accuracy as the pruning rate rises while the model size, parameter count, and multiply-accumulate count all stay unchanged, because the operation zeros the weights rather than removing them (Francy & Singh, 2024). A zeroed weight still occupies memory and still enters the dense matrix product unless the backend has a sparse kernel that skips it. Mei and Stamp trained nine convolutional models under global unstructured pruning at ratios from 0.27 to 0.61 of the original weights, and found that neither pruning nor low-rank factorization produced consistent energy reductions, increasing energy in several cases, and that the pruned models often needed more training epochs than the baselines (Mei & Stamp, 2025). Their nominal-versus-realized gap is about retraining cost rather than backend execution, but the lesson runs in the same direction: removing parameters does not automatically remove work.

Structured pruning removes work by design, and its energy behavior depends on what replaces the removed units. Dobler and Lemmerich compress networks by merging almost-linear neurons into skip connections and report that model sizes fall by about ten percent without performance change and by up to a third with a small loss (Dobler & Lemmerich, 2025). They are explicit that the compression is not free at runtime: the method reduces parameters while the more complex inference function requires more steps and more temporary memory, and they list the reduction of inference time as unaddressed (Dobler & Lemmerich, 2025). Francy and Singh contrast the two regimes directly, reporting structured pruning that cuts parameters by sixty-one to seventy-five percent and multiply-accumulates by forty-five to sixty percent with no accuracy drop (Francy & Singh, 2024). The comparison is the clearest statement in the literature that the pruning method, not the compression ratio, determines whether an energy saving exists to be collected.

The backend decides whether a nominal saving becomes a realized one. An unstructured model with no sparse kernel executes the same dense computation as its original, and a structured model whose framework lacks optimized layers for the new shapes can run slower than the dense baseline it replaces. The energy consequence of a pruning method is therefore a property of the model, the runtime, and the hardware together, and a sparsity figure quoted alone names an intention rather than a result. Chang and colleagues provide the cleanest measurement of this gap. Unstructured pruning left every ResNet50V2 variant at exactly 7.76 GFLOPs, so the correlation between FLOPs and energy was formally undefined, yet measured energy at identical FLOPs varied from -11.8 to -0.6 percent on the CPU INT8 delegate, from -6.6 to -3.5 percent on GPU INT8, and from -7.6 to $+17.2$ percent on TPU INT8 (Chang et al., 2026). A proxy that does not move cannot explain an energy change that does.

Kim and colleagues' LayerMerge illustrates the same lesson from the opposite direction: a compression method evaluated on latency but not energy. On MobileNetV2-1.4 for ImageNet, their layer-pruning-and-merging method reaches a 2.61-fold latency speed-up at 73.48 percent accuracy, against 2.27-fold at 73.46 percent for an earlier depth-compression method (Kim et al., 2024), but the paper measures no energy, so

whether the latency gain and an energy gain move together for this method, as Section 3.4 shows they often do not, is a question it does not ask.

4.3 Knowledge Distillation

Knowledge distillation trains a smaller student model to reproduce the outputs of a larger teacher, so the compressed artifact is a smaller model rather than a sparsified or quantized version of the original. Hernandez and colleagues distil a MobileNetV2 teacher into a depth- and width-reduced student, achieving a 4.1-fold parameter reduction to 24.3 percent of the teacher's size while preserving 98.4 percent of its accuracy, and measure a tenfold inference speedup from 140 to 13 milliseconds (Hernandez et al., 2025). They attribute the speedup, which exceeds the parameter reduction, to reduced memory-access patterns and better cache utilization, a memory-side effect consistent with Section 3.

Distillation's energy benefit is indirect, and its cost is usually unmeasured. The student is a conventional small network, so its inference energy follows the same hardware and batch-size dependencies as any other model, and the technique guarantees nothing about energy beyond the fact that the network is smaller. The distillation process itself consumes energy: training the student against the teacher's outputs, and any preprocessing such as the integrated-gradients attribution maps that Hernandez and colleagues precompute, is a training cost that the paper describes as a one-time step without quantifying it (Hernandez et al., 2025). A distillation study that reports inference latency but not training energy leaves the technique's net energy balance unstated.

Distillation is also rarely compared against quantization and pruning on equal footing. Comparative work tends to place it alongside the other methods within a single study, which is where the interaction effects become visible and where distillation's reputation for balance is either confirmed or qualified (Paula et al., 2025).

4.4 Combined Approaches

Production systems stack compression techniques, and the interaction between them is not additive. Paula and colleagues compared pruning, distillation, and post-training quantization individually and in combination across four transformer models, measuring energy with CodeCarbon and reporting savings from 2.62 to 91.26 percent relative to a BERT baseline (Paula et al., 2025). Two results show that the combination can move against expectations. DistilBERT with pruning consumed more energy than the unpruned model, which the authors attribute to storage and computational inefficiency from the index handling that irregular sparsity introduces. ALBERT with quantization collapsed in accuracy to 65.44 percent, which they trace to precision loss in an already highly compressed architecture whose self-attention ranking is disrupted by quantization.

The same study concludes that each technique's effectiveness depends heavily on the model architecture and application constraints, and that quantization may be unsuitable for an already compressed architecture (Paula et al., 2025). A different combination moved the other way: Francy and Singh combined structured pruning with dynamic eight-bit quantization and reported an 89.7 percent size reduction with accuracy improving by 3.8 percent, a near-compounding result they enabled by keeping the pruned model in a format the quantizer could consume (Francy & Singh, 2024). The two studies differ in model family and pipeline, and together they support a single conclusion: whether techniques compound, cancel, or interfere depends on the model and the order in which they are applied, and the outcome cannot be predicted from the individual compression ratios.

Xiang's device-aware framework combines structured compression, knowledge distillation, and mixed-precision quantization inside a differentiable, hardware-aware neural architecture search. It reports that the resulting configuration reaches 79.7 percent top-1 accuracy on ImageNet-1k while cutting latency from 84 to 58 milliseconds and energy from 505 to 330 millijoules relative to a strong post-training-quantization baseline (Xiang, 2026). On CIFAR-10 and Speech Commands, the same framework reaches 95.7 percent accuracy at 14.2 milliseconds and 78 millijoules, and 95.1 percent at 4.3 milliseconds and 19 millijoules, respectively (Xiang, 2026). Unlike the combination Paula and colleagues report above, this one moves latency and energy in the same direction at the same time, which the search procedure achieves by optimizing for hardware cost directly rather than assuming a proxy metric predicts it.

Three further combination studies illustrate how much of this literature stops at accuracy and compression ratio without reaching energy at all. Makenali and colleagues integrate similarity-based filter pruning with

power-of-two quantization, either simultaneously during training or in sequence, and report effective compression with minimal accuracy loss but no energy or latency measurement (Makenali et al., 2025). Schulze and colleagues' PQ Bench automates benchmarking of pruning and quantization combinations across accuracy, inference speed, and memory footprint, a genuine advance in tooling that stops one axis short of the question this review asks (Schulze et al., 2025). Zandonati and colleagues' FITCompress unifies mixed-precision quantization and unstructured pruning through a Fisher-information-guided search and reports a superior compression-performance trade-off on vision and language benchmarks, again without an energy figure; they chose bit-operations as their efficiency measure because, they argue, latency and power depend on specific kernel implementations and can misrepresent compression (Zandonati et al., 2023). That is the opposite inference from the one this review draws, since the kernel dependence that led them away from measured energy is exactly what a deployment decision has to confront. Three independent groups built genuinely more sophisticated joint-compression methods, and three papers that could have measured whether the sophistication pays for itself in energy did not; that gap is itself a data point about where the field's attention has been.

4.5 The Accuracy-Efficiency Frontier

The compression techniques do not share a common currency. One reduces memory, another reduces operation count, another reduces latency, and each moves accuracy by a different amount, so choosing among them for a deployment means trading accuracy against an efficiency metric that differs by technique. Rajput and Sharma compared five four-bit post-training quantization methods on a single language model and found total energy ranging from 308 to 1,123 milliwatt-hours across the methods, with the most efficient method more than twice as efficient as the least at the same nominal bit width (Rajput & Sharma, 2024). They conclude that energy advantages depend on model architecture and layer distribution, and that no single technique optimizes all scenarios.

Independent measurement attributes the variation to how well the workload uses the hardware, not to the compression ratio. Fanariotis and colleagues measured per-inference energy for an accelerator against a microcontroller core and found speedups from about seven to over 126 times, with one small model regressing to 0.62 times because its utilization of the accelerator was only 5.1 percent, so the orchestration overhead outweighed the acceleration (Fanariotis et al., 2025). A compressed model that leaves the accelerator idle can cost more energy than the dense model that kept it busy, which is the same utilization mechanism that governs batch size.

Within the corpus (Section 1.4), no study constructs a formal Pareto frontier of accuracy against measured energy; the closest approximation is an energy-versus-accuracy scatter whose authors note diminishing returns (Paula et al., 2025), and the supplementary search located formal accuracy-efficiency frontiers for compressed models only over latency and model size. Across the corpus, the best compression state is architecture-, hardware-, and batch-size-dependent, so no study in it offers a single recommended configuration for a given accuracy target. The choice has to be measured for the deployment at hand, and the next section shows that the measurement itself changes with the platform.

5 Cross-Platform Energy Behavior

A compression result is only portable if the hardware it was measured on resembles the hardware it will be deployed on. This section reviews what is known about energy behavior on the four platform classes that dominate machine learning deployment, x86 CPUs, ARM processors, GPUs, and emerging accelerators, and then asks whether the ranking of compression strategies by energy efficiency survives a change of platform (Gap 3, Section 7.2).

5.1 x86 CPU Energy Profile

Server, desktop, and laptop x86 processors are the platform on which software-based energy measurement is most established, because RAPL exposes processor energy through a documented interface (Section 2.2). The interface is not uniform across vendors: on AMD processors only the package and core domains are exposed, with no DRAM or uncore counters, so reading the package domain on an AMD system captures a different quantity than the same code reads on an Intel system (Raffin & Trystram, 2024). Access is also constrained in practice. Dos Santos Ferreira and colleagues note that unprivileged access to RAPL under Linux was restricted in 2020 in response to a security vulnerability, so measuring CPU energy typically requires root access (Dos Santos Ferreira et al., 2026).

Measured x86 results show how much of an inference workload’s energy the host processor carries even when an accelerator does the arithmetic. Alizadeh and Castor measured CPU energy through RAPL and GPU energy through `nvidia-smi` on a laptop with an Intel Core i7-11850H and an RTX 3070, subtracting idle baselines for both, and found mean CPU energy equal to 51.71 percent of mean GPU energy across all executions, and 38.43 percent after removing outliers (Alizadeh & Castor, 2024). The share depends strongly on the framework: for MobileNet at batch size one under TensorFlow, the CPU consumed 4,457.76 J against 610.5 J for the GPU (Alizadeh & Castor, 2024). On x86 systems a GPU-only reading can therefore miss most of the energy, and the measurement boundary is part of the result. For training, Sierra-Herrera and colleagues found that general-purpose CPUs showed significant scalability limitations for deep learning and that performance varied with batch size most strongly on CPU-based platforms (Sierra-Herrera et al., 2026), which extends the batch-size dependence of Section 3.3 to the host processor.

5.2 ARM CPU Energy Profile

ARM processors dominate mobile and edge deployment, and they are the hardest platform on which to measure energy. The interface that anchors x86 measurement does not exist on ARM: a survey of on-device techniques states that the available tools work on a limited number of hardware platforms, generally restricted to Intel CPUs and NVIDIA GPUs, and that where a tool runs out of supported hardware it falls back to a constant thermal-design-power estimate that is not accurate (Boumendil et al., 2024). The countervailing measurement options are the ones reviewed in Section 2: on-device sensors calibrated against an external reference, the power management integrated circuit, a platform-level interface such as Apple powermetrics, or an external meter attached to the power rail.

The ARM power envelope is smaller and its thermal behavior more aggressive, which changes the shape of an energy measurement rather than only its scale. Tummalapalli and colleagues ran a four-bit language model under sustained load across an ARM single-board computer, a flagship Android phone, and an iPhone, and quantified the throttling directly. The iPhone lost about forty percent of its peak throughput within three iterations and settled at a plateau of 23.67 tokens per second, while the Android device degraded by about fifteen percent and held around 10.38 tokens per second after its clock settled into a lower band (Tummalapalli et al., 2026). On a passively cooled device, a short benchmark and a sustained workload measure different power profiles.

ARM systems also constrain the memory path more tightly than desktop platforms. The same study found the decode phase memory-bandwidth bound, with the on-module memory of an attached accelerator setting the ceiling and the physical link between the host and the accelerator further reducing achievable bandwidth (Tummalapalli et al., 2026). Memory-bound behavior that is visible on x86 becomes binding on ARM, which sharpens the effect of the mechanisms in Section 3 and makes the energy consequence of a compression technique depend on whether it relieves memory pressure or compute pressure. Reviews of lightweight models for edge computing reach the same structural conclusion without the same measurements: they frame the design problem as hardware-aware, with latency and power on the target hardware entering as constraints rather than as afterthoughts (Lai et al., 2026).

Giedra and colleagues add a second data point at the same scale as Tummalapalli’s, benchmarking twelve compact and mid-scale open-weight language models from under one billion to eight billion parameters on an NVIDIA Jetson Orin Nano Super, combining internal tegrastats rail readings with an external digital multimeter at the board input, and reporting throughput, power, and energy alongside generation-quality accuracy, dimensions that are more commonly reported in isolation (Giedra et al., 2026). Their joint accuracy-efficiency results are reported per model rather than as the formal frontier over measured energy that Section 4.5 finds absent from the corpus, and their restriction to a single edge board is the cross-platform limitation Section 5.5 identifies.

5.3 GPU Energy Profile

GPUs pair high absolute power with high memory bandwidth, and the balance makes their energy behavior unusually sensitive to how fully the workload uses the parallel units. Embedded boards show the power cost of engaging the GPU at all. Lahmer and colleagues modeled the energy of convolutional and fully connected layers for CPU inference on two NVIDIA Jetson boards, and observed that the boards’ peak and average power were lower when inference ran on the CPU than when it used the GPU (Lahmer et al., 2022). Their per-board model rests on the boards’ onboard power sensors, read in 0.1 ms slots, and later work from an overlapping group of authors found that onboard sensors on Jetson devices, including the same TX2 and

Xavier NX boards, under-report actual power by up to 50 percent before calibration (Shalavi et al., 2023), so the absolute values of energy models built on uncalibrated onboard readings inherit that bias.

The utilization dependence is what makes batch size decisive on GPUs. Tu and colleagues measured that a convolution on a mobile GPU could consume more than 6.6 times the energy of the same operation on the CPU at small feature sizes, while at the model level the GPU saved between 49 and 79 percent of energy relative to the CPU (Tu et al., 2023). A GPU earns its efficiency when a large workload keeps its parallel units busy, and works against the energy budget when a small workload leaves them idle. This same mechanism can make a compressed model consume more energy than its dense original.

Scaling the number of GPUs shows the same utilization mechanism working against intuition. Chung and colleagues found that adding GPUs at a fixed batch size increased energy per token in 81 percent of their B200 configurations and 93 percent of their H100 configurations, because splitting the same batch across more accelerators lowers the compute each one performs without lowering its idle and communication overhead (Chung et al., 2026). Energy fell only when the additional GPUs unlocked a larger batch size than a single device's memory could hold, so the extra hardware was repaid in utilization rather than spent on communication. Whether more hardware helps or hurts energy is therefore not a property of the hardware count alone but of whether it changes the batch size the workload can run.

Utilization-driven power management is not limited to compression; it operates directly on the accelerator's operating point. Yuan and colleagues' KAIROS jointly manages GPU frequency, per-instance concurrency, and request placement for agentic large-language-model serving, reducing average GPU power by 27 percent, and by up to 39.8 percent, for a single serving instance and by 46.3 percent for multi-instance serving while meeting throughput targets (Yuan et al., 2026). Their characterization shows why the knob is not monotonic: lowering GPU core frequency from 1,680 to 900 MHz reduced energy and power by about 30 percent with little effect on throughput or latency, but below 900 MHz energy rose sharply as slower execution let pending work accumulate (Yuan et al., 2026). The mechanism is the same utilization sensitivity this section documents for compression: memory headroom and demand, not the size of the reduction, decide whether power can be recovered without cost.

5.4 Emerging Hardware: NPUs, TPUs, and Neuromorphic Devices

Specialized accelerators enter the literature with large efficiency claims and comparatively little independent measurement. Neural processing units in embedded microcontrollers are the best-measured case. Fanariotis and colleagues compared an accelerator against a microcontroller core directly, using an external multimeter to measure per-inference net energy, and reported speedups from about seven to over 126 times and energy reductions up to 143 times, with one small model regressing to 0.62 times because its utilization of the accelerator was only 5.1 percent (Fanariotis et al., 2025). Their conclusion scales the benefit with utilization and model complexity rather than with the accelerator's presence.

A second accelerator case places an NPU alongside a phone and a discrete GPU within one comparative study. Tummalapalli and colleagues measured an NPU-attached single-board computer at 0.271 watts per token per second and a laptop GPU at 0.259, despite a nineteen-fold gap in absolute throughput (Tummalapalli et al., 2026). The authors stress, however, that the two figures are not directly comparable: the NPU value is whole-system draw, while the GPU value is GPU-level power that excludes the rest of the laptop, and they limit their own claim to the NPU not trading significantly worse energy per token for its low power envelope rather than achieving GPU-class efficiency (Tummalapalli et al., 2026). Read with that qualification, the comparison still undercuts the expectation that a dedicated unit delivers energy savings proportional to its speedup, without establishing that the two accelerators are equally efficient.

Vendor-supplied accelerator figures are common, and independent verification is rare. Even Fanariotis and colleagues, whose own results come from direct multimeter measurement, frame the expected efficiency of microcontroller NPUs in their background with figures they do not measure, stating that NPUs deliver on the order of 1 TOPS/W and citing benchmarking studies that report more than 3 TOPS/W (Fanariotis et al., 2025). The same pattern applies to mobile neural engines and tensor processing units, whose headline efficiency numbers appear in specifications more often than in third-party measurements. Neuromorphic hardware, whose event-driven design promises an energy advantage over clocked computation, remains mostly theoretical at deployment: a survey of on-device techniques describes implementations on research platforms such as Intel Loihi and the ARM-based SpiNNaker without reporting validated deployment-scale

energy savings (Boumendil et al., 2024). This review treats these claims consistently: they are inputs to be verified rather than results to be cited.

A counterexample exists at the model level, short of full deployment validation. Li, Gao, and colleagues' spiking object detector reports 6.43 millijoules per inference alongside a mean average precision of 62.0 percent on COCO and 66.3 percent on Gen1, and an 82.9 percent energy reduction relative to the baseline network in their Gen1 ablation (Li, Gao et al., 2025). That energy figure is estimated from operation counts, accumulate and multiply-accumulate operations weighted by firing rate and time steps, rather than measured on hardware (Li, Gao et al., 2025), and it is model-level energy on a research benchmark, not the on-device, deployment-scale figure this subsection's opening claim describes as absent. The two claims are therefore compatible: spiking networks are quantified more often at the algorithm level than at the deployment level, the algorithm-level figure is itself an operation-count proxy of the kind Section 3 examines, and the gap between the two is a version of the nominal-versus-realized distinction Section 7.2 names.

5.5 Cross-Platform Rank-Order Consistency

The question that decides whether a compression result generalizes is whether the ranking of strategies by energy efficiency is the same on every platform. The evidence available suggests it is not. Sierra-Herrera and colleagues benchmarked training across x86 CPUs, NVIDIA GPUs, Apple Silicon, and a Raspberry Pi with a fixed methodology and reported the speed and efficiency rankings separately. The fastest platform was not the most efficient: an Apple M4 GPU ranked third or fourth in speed while placing highest in energy efficiency, and for one model family the Apple M4 CPU was the most energy-efficient platform of all, ahead of every GPU tested (Sierra-Herrera et al., 2026). Their overall conclusion is that hardware efficiency is strongly model-dependent, meaning the ranking is a property of the model-platform pair rather than the hardware alone.

A second study reaches a compatible result by a different route. Tummalapalli and colleagues found an NPU and a discrete GPU with nearly the same energy per throughput (Tummalapalli et al., 2026), so the faster accelerator is not necessarily the more efficient one, and the ordering depends on which quantity the deployment cares about. The same study issues the caution that governs this whole section: energy figures are not directly comparable across platforms because the measurement scope differs, with a whole-system rail reading, a GPU-level reading, and a fuel-gauge reading each capturing a different boundary (Tummalapalli et al., 2026). A rank order assembled from measurements at different scopes may reflect the scopes more than the hardware.

Two further results show the platform deciding the outcome of a compression comparison. Francy and Singh found that identical structured pruning of the same architecture produced different accuracy depending on which implementation of the architecture was pruned, at 63 versus 73 percent (Francy & Singh, 2024). Soltani and colleagues, estimating edge-device energy for a split-learning deployment, found the technique's efficiency held for lightweight models, while communication and memory overhead reduced the gain for deeper networks (Soltani et al., 2025). In each case, the sign of the benefit depended on the model-hardware combination rather than on the compression method alone. The literature's consistent message for cross-platform work is that a claim of universal efficiency, stated without the platform it was measured on, cannot be checked on another platform and should be read as a statement about one system.

6 Deployment Conditions That Benchmarks Do Not Reproduce

Benchmarks are usually short, controlled, and run on mains power. Deployment is long, uncontrolled, and often on a battery. This section reviews three factors that separate the two settings: thermal throttling on passively cooled devices, the power delivery of a discharging battery, and the handling of idle baselines. The corpus shows that each factor can change measured energy; whether each changes the energy advantage of a compressed model specifically follows by construction for idle baselines, is supported only indirectly for thermal state, and is not measured at all for battery state.

The gap between benchmark and deployment is not limited to energy figures; it appears first in raw performance. Mishra and Lone measured continuous, sustained roadside object detection on a Jetson Orin Nano across three state-of-the-art baselines and found a consistent 20 to 30 percent relative degradation when moving from static-image benchmark evaluation to real-world streaming deployment, under deployment effects that include temporal instability in streaming video and thermal throttling under sustained load (Mishra & Lone, 2026). Their temporal-stabilization mechanism recovered up to 10.16 percent of

classification accuracy over per-frame baselines while sustaining real-time operation, which shows the degradation is partly an artifact of the evaluation protocol rather than an unavoidable cost of sustained operation (Mishra & Lone, 2026). The same distinction, between what a benchmark measures and what a sustained deployment experiences, is this section’s subject for energy rather than accuracy.

6.1 Thermal Throttling on Passive-Cooled Devices

Mobile and edge devices dissipate heat through a passive enclosure, so sustained inference raises the temperature until the device reduces its own frequency to stay within a thermal limit. Zou and colleagues measured this on an on-device language model and found that a shell temperature of 42 degrees Celsius is reached within a hundred seconds of continuous decoding and 45 degrees within two hundred, as autoregressive decoding holds the accelerator under load for tens of seconds to minutes at a time (Zou et al., 2026). Their stress test showed the practical consequence: a default configuration generated only 690 tokens in forty seconds before crossing the threshold, while their thermal-aware controller sustained 1,784 tokens over 165 seconds, 27.9 percent more tokens over a 32.1 percent longer session (Zou et al., 2026).

The temperature dependence reshapes the energy figure rather than scaling it. Because a throttled device runs at a lower frequency and a different power draw, energy is not a monotonic function of time or of the work completed: the same workload split across two sessions meets a different thermal state than the same workload run continuously. Tummalapalli and colleagues observed the same effect across two phones and quantified the throughput plateau, with one device losing about forty percent of its peak within three iterations and settling at a fixed temperature (Tummalapalli et al., 2026). A short benchmark that finishes before the temperature rises reports the unthrottled numbers, and a sustained deployment reports the plateau.

The benchmark-to-deployment gap becomes a methodological warning. Zou and colleagues distinguish bursty or intermittent workloads from the sustained load of long-form generation and argue that configurations ignoring thermal dynamics fail to maintain comfortable temperatures under sustained use (Zou et al., 2026). TinyML benchmarking work makes the adjacent point that phase isolation and measurement uncertainty are rarely reported, so a method that captures inference in a cold steady state may not represent a device that runs continuously (Bartoli et al., 2025). Compression interacts with this setting in a way the literature has not settled: a quantized model that draws less power may heat a passive device more slowly and delay throttling, but no study in the corpus measures the interaction directly. The clearest evidence that this interaction would matter, if measured, comes from delegate-specific pruning effects rather than a direct thermal study. Chang and colleagues profiled quantized and pruned ResNet-50 variants across CPU, GPU, and TPU delegates on a Pixel 9 and found that the sign of pruning’s energy effect flipped with both delegate and pruning location: under CPU FP32, early-stage pruning gave the largest savings while stem pruning cost energy; under GPU INT8, pruning gave overall reductions; and on the TPU, late-stage and global pruning helped while stem- and middle-stage pruning increased consumption (Chang et al., 2026). A pruning method that increases power draw on one delegate while reducing it on another is exactly the kind of delegate-specific behavior that would interact with the thermal dynamics this subsection describes, yet neither this study nor any other in the corpus measures the two together (Gap 6, Section 7.2).

6.2 Battery Discharge and Power Delivery

Battery power delivery is nonlinear, and the energy measured under mains power is not the energy drawn under a discharging cell. As a battery discharges, its voltage falls, and the device power-management subsystem responds by adjusting frequency and voltage limits, so the same workload meets a different power state at twenty percent charge than at full charge. A sustained inference session therefore draws from a supply whose characteristics change over the session, and thermal and battery effects compound: low charge reduces the voltage ceiling at the same time a warm enclosure reduces the frequency ceiling.

The corpus treats battery state as a limitation rather than a variable to study. Tummalapalli and colleagues measured one phone through its battery-management interface and reported energy from a fuel gauge, which captures the device-level boundary including battery behavior, while noting that energy figures from their platforms are not directly comparable because their measurement scopes differ (Tummalapalli et al., 2026). The same study ran its laptop GPU on battery and found sustained throughput bounded by the battery power ceiling, which shows that the supply can be the binding constraint during inference (Tummalapalli et al., 2026). Husom and colleagues, profiling on a single-board computer, measured idle power separately at 2.85 watts and subtracted it, a protocol that assumes a stable supply across the session (Husom et al., 2025). The structure of the problem raises the question of whether a compression ranking measured on mains power

holds on a discharging battery, and no study in the corpus or located by the supplementary search answers it (Gap 5, Section 7.2). The reason is practical: controlling and instrumenting battery state reproducibly is harder than holding a device on a fixed supply.

6.3 Idle Baseline and the Comparability Problem

Every energy measurement must decide what belongs to the workload and what belongs to the device. These choices fall into three conventions used in the literature. The first subtracts a measured idle baseline to report net task energy: Husom and colleagues measured idle power at 2.85 watts with a standard deviation of 0.17 watts over 104 minutes and removed it from the inference figure (Husom et al., 2025), and Rajput and colleagues waited for a stable baseline before each API call and subtracted it as well (Rajput et al., 2023). The second reports total system energy without subtraction, which is the natural output of a wall meter and the convention of a reproducible benchmarking framework that measures power under consistent supply conditions without isolating the idle component (Sierra-Herrera et al., 2026). The third reports energy per unit of work, such as joules per inference or per token, which normalizes for the amount of work and absorbs the baseline into the per-unit figure.

The three conventions do not produce the same number, and the difference is not small when the baseline is a fixed cost against a short workload. A device that idles at nearly three watts and completes an inference in tens of milliseconds carries an idle contribution that competes with task energy at the granularity where fine-grained attribution matters (Husom et al., 2025). A reader comparing a net figure from one paper with a gross figure from another, or a per-inference figure with a per-token figure, is comparing quantities that differ by a definitional constant before any hardware difference enters. The standardization gap described in Section 2 is visible here in its most concrete form: the same measurement, reported under two conventions, supports two different conclusions about how much energy a compressed model saves.

The dynamic factors of this section share a common structure. Each one—thermal state, battery state, and idle baseline—is a variable that the benchmark fixes and the deployment lets move, and each can move the energy figure by an amount comparable to the savings that compression claims. A methodology that holds these variables constant can measure a compression effect precisely and still mispredict the deployment, which is why a measurement protocol and the conditions it controls are as much a part of an energy result as the hardware it runs on.

7 Gaps, Contradictions, and Open Questions

The preceding five sections describe a field that agrees on the importance of energy and disagrees on how to measure it. This section gathers the points where the literature conflicts with itself, names the gaps those conflicts expose, and states the open questions they leave for future work.

7.1 Contradictions in the Literature

The most consequential contradiction concerns the reliability of the software tools that most studies rely on. Fischer ground-truthed CodeCarbon against external meters and found a systematic underestimate of 15 to 25 percent (Fischer, 2026), while other studies cite its use without reporting a validation error, and still others report the tool as adequate for their purposes. A tool cannot be simultaneously biased by a fifth and accurate to within a few percent, so the disagreement reflects the conditions under which each assessment was made: the hardware, the workload, and the presence or absence of an external reference. Diamond and Stoico sharpen the point from the overhead side, showing that the same class of tools imposes time overheads from 0.25 to 46.75 percent, depending on the sampling rate and the benchmark (Diamond & Stoico, 2026). The measurement tool and the workload together determine the error magnitude, and a study that reports neither has reported an energy number of unknown accuracy.

A second contradiction concerns whether quantization saves energy. The analytic case is that INT8 reduces matrix-multiplication cost by a factor of sixteen (Nagel et al., 2021), and comparative studies report energy savings up to 91 percent relative to an unquantized baseline (Paula et al., 2025). Against this, the same comparative study finds pruning combined with a model consuming more energy than the unpruned version (Paula et al., 2025), a quantized model collapsing in accuracy to 65.44 percent (Paula et al., 2025), and the general finding that activation-only quantization fails to accelerate inference because the bottleneck is weight access rather than activation memory (Li et al., 2024). The resolution is that quantization saves energy when the arithmetic is the bottleneck and the backend executes the reduced-precision path, and neither condition is guaranteed.

A third contradiction concerns pruning. The claim that removing half the filters removes half the work holds for structured pruning under a backend that supports the resulting shapes, as Francy and Singh found when structured pruning cut parameters by sixty-one to seventy-five percent with no accuracy loss (Francy & Singh, 2024). The same authors found that unstructured pruning left model size, parameter count, and operation count unchanged because it zeros weights rather than removing them (Francy & Singh, 2024), and Mei and Stamp found pruning failing to reduce energy and sometimes increasing it because the pruned models needed more training epochs (Mei & Stamp, 2025). The two findings describe different methods under different conditions, and a report that names only the compression ratio cannot distinguish them.

7.2 Gaps in the Current Literature

The contradictions organize into six gaps. Each could be addressed by a single study, and within the corpus and the supplementary search described in Section 1.4, none has been closed. Table 4 maps each gap to the studies that expose it.

The first is a systematic validation matrix for measurement tools on compressed models. Section 2 showed that software estimators, hardware counters, and external meters disagree by margins comparable to the compression savings they measure, and that direct validation is rare. What is missing is a study that places CodeCarbon, a RAPL-based counter, and an external meter on the same compressed models across the same platforms so that tool error can be attributed to precision rather than inferred from unrelated experiments. Fischer's ground-truthing study is the closest precedent and is limited to a single platform (Fischer, 2026); an extension to quantized and pruned models does not appear in the corpus. Such a study would also need to report the accuracy class, sampling rate, and measurement boundary of its reference meter, for the reasons given in Section 2.3.

The second is the gap between nominal and realized compression, which is the compression-specific form of the FLOPs-energy disconnect in Section 3. A model described as INT8 or thirty-percent pruned carries a nominal compression, counted in bit widths, sparsity, or operations, that the deployment may not realize, because the backend may run dense computation, fall back to a wider format, or spend the saved arithmetic on conversion overhead. Li and colleagues documented the de-quantization step back to FP16 (Li et al., 2024), Dobler and Lemmerich documented a compression that reduced parameters while increasing inference steps (Dobler & Lemmerich, 2025), and Francy and Singh documented unstructured pruning that changed nothing at runtime (Francy & Singh, 2024). The strongest single demonstration is Chang and colleagues' finding that FLOPs stayed fixed at 7.76 GFLOPs across unstructured-pruning variants while measured energy moved by between -11.8 and +17.2 percent across INT8 delegates (Chang et al., 2026). The mechanisms of Section 3, memory bandwidth, batch size, and backend realization, are the candidate explanation for the difference. A method that measures realized compression directly, by instrumenting what the backend executes rather than what the model file reports, would test that explanation, and no such method appears in the corpus.

The third is cross-platform rank-order preservation. Section 5 found that hardware efficiency is model-dependent and that the fastest platform is not the most efficient (Sierra-Herrera et al., 2026). However, that comparison rests on training benchmarks, the multi-platform studies in the corpus compare a handful of devices each, and their measurement boundaries differ (Tummalapalli et al., 2026). Whether the ranking of compression strategies by inference energy is preserved from x86 to ARM, or from a mobile CPU to an accelerator, is untested in the corpus. What scale would be sufficient to settle the question in general is itself open (Section 7.4). The gap that a single study can address is narrower: a protocol-matched comparison in which the models, compression states, instruments, and reporting conventions are held fixed, so that any change in ranking can be attributed to the platform rather than to differences in protocol.

The first three gaps are the ones a single controlled study, of the kind outlined in Section 8.3, can address directly, within the scope limits stated there. The remaining three name real gaps but require instrumentation, access, or time beyond that scope, and are noted here as future work rather than as part of that agenda.

The fourth is validating on-device power measurement. The PMIC-based framework on a mobile platform achieved low internal error, but it was validated against a direct integration of the same trace rather than an independent meter (Chang et al., 2026), and the Jetson internal sensors underestimated power by up to fifty percent before calibration (Shalavi et al., 2023). Because mobile energy studies rely on these interfaces, an

unvalidated PMIC reading puts every downstream mobile result on uncertain ground, and a systematic calibration against an external meter does not appear in the corpus.

The fifth is compression behavior under battery power. Section 6 showed that battery state changes the supply a device draws from and that no study in the corpus measures a compression ranking on a discharging battery. If the ranking shifts with charge state, then the mains-power measurements that dominate the literature may not transfer to the mobile deployments they are meant to inform.

The sixth is the interaction between thermal behavior and compression. Zou and colleagues studied thermal-aware control for language models (Zou et al., 2026), and the delegate comparison showed compression energy effects varying by execution unit (Chang et al., 2026), but no study in the corpus measures whether a quantized model, by drawing less power, delays thermal throttling enough to change its own energy advantage over a sustained run. The question sits at the intersection of two lines of work that have not yet met.

Table 4. Gap-to-evidence matrix for the six gaps in Section 7.2.

Gap (Section 7.2)	Corpus studies that expose it	What they show	What is missing
1. Tool validation on compressed models	Fischer (2026); Aquino-Brítez et al. (2025); Diamond & Stoico (2026); Sardi et al. (2026); Caravaca et al. (2025)	Estimator bias, overhead, and learned-model error measured on uncompressed workloads	Estimator, counter, and meter run in parallel across compression states
2. Nominal vs. realized compression (FLOPs-energy mechanism)	Li et al. (2024); Dobler & Lemmerich (2025); Francy & Singh (2024); Chang et al. (2026); Chung et al. (2026); Li, Paolieri & Golubchik (2025)	De-quantization, extra inference steps, unchanged FLOPs with changing energy, FP8 losing at small batch, slower integer kernels	Instrumentation of what the backend executes, related to the mechanisms of Section 3
3. Cross-platform rank-order preservation	Sierra-Herrera et al. (2026); Tummalapalli et al. (2026); Giedra et al. (2026); Francy & Singh (2024); Soltani et al. (2025)	Rankings tied to the model-platform pair; measurement scopes that differ across platforms	Compression rankings measured under one protocol across platform classes
4. On-device power validation	Chang et al. (2026); Shalavi et al. (2023); Lahmer et al. (2022)	PMIC validated only against an internal reference; onboard sensors under-report by up to 50% before calibration	PMIC readings calibrated against an external meter
5. Compression under battery power	Tummalapalli et al. (2026); Husom et al. (2025)	Battery treated as a limitation; fuel-gauge scope; stable-supply assumption	Compression rankings measured across states of charge
6. Thermal behavior and compression	Zou et al. (2026); Tummalapalli et al. (2026); Mishra & Lone (2026); Chang et al. (2026)	Throttling within minutes of sustained load; delegate-specific pruning effects	Thermal trajectory of compressed and dense models under matched sustained load

7.3 Reproducibility

Energy results are hard to reproduce for reasons specific to the measurement rather than the model. Datasets and models are often public, but the measurement harness usually is not, so a reader cannot rerun the experiment that produced a reported figure. Protocols differ on every axis that matters: whether the idle baseline is subtracted, whether the first run is discarded, what batch size is fixed, how many runs are averaged, and what dispersion is reported. Hardware descriptions often omit driver and library versions and the runtime backend, which determine whether a reduced-precision path runs at all. As a result, each study re-invents its methodology, and two studies of the same compression technique can disagree because their protocols differ before their hardware does. The standardization efforts described in Section 2 address this problem by prescribing a protocol (Tschand et al., 2024), and their existence alongside continued variation in practice marks the gap between a proposed standard and an adopted one. Box 1 collects the reporting items that Sections 2.3, 2.4, and 6.3 and this section identify as decisive, as a minimum that would make an energy figure comparable across studies.

Box 1. Minimum reporting checklist for energy measurements of compressed models
1. Instrument, with accuracy class, sampling rate, and measurement boundary (component, board, or wall).
2. Idle-baseline convention (net, gross, or per unit of work), with the idle power value where subtracted.
3. Warm-up handling: whether initial runs were discarded and how steady state was identified.
4. Batch size for every energy figure and, for language models, input and output lengths.
5. Runtime, backend, driver, and library versions, and whether the reduced-precision or sparse kernel path was executed.
6. Nominal compression (bit width, sparsity, FLOPs) together with evidence of realized compression.
7. Thermal state, session length, and ambient temperature for passively cooled devices.
8. Power source (mains or battery) and state of charge for battery-powered devices.
9. Number of repetitions and dispersion (standard deviation or confidence interval).
10. Measurement overhead of the instrumentation, and availability of the measurement harness.

7.4 Open Questions

Five questions follow from the gaps. Can an energy measurement harness be built that is portable across hardware classes without losing the access to power that each class provides in its own way? What is the

smallest set of platforms that establishes whether a compression ranking generalizes, given that efficiency is model-dependent and the ranking is a property of the model-platform pair? How does thermal behavior modify the energy profile of a compressed model over a sustained workload? Does the accuracy a compressed model retains change its position on the efficiency frontier, so that the cheapest models to run are also the least accurate, or do the two trade against each other differently than expected? And for language-model inference, is the asymmetry between the compute-bound prefill phase and the memory-bound decode phase stable across model size and quantization, or does it change with both? Each question can be answered with the methods this review surveys, and none has yet been answered with them.

8 Synthesis and Positioning

8.1 What the Literature Establishes

Four results are supported well enough to guide practice. Computation is not a reliable proxy for energy: the correlation between operation count and measured energy is weak across model families, and memory bandwidth, arithmetic intensity, and hardware utilization break the correspondence. Compression reduces energy when the backend executes the reduced computation and the workload uses the hardware well, but the realized reduction is often smaller than the nominal compression ratio suggests. Efficiency rankings vary across platforms to the point that the fastest platform is often not the most efficient, so no universal best configuration exists. Thermal and idle-baseline effects can shift the energy figure by amounts comparable to the compression savings, making the measurement protocol part of the result.

8.2 What Remains Uncertain

Three questions remain open despite the volume of work surveyed. Which measurement tool is most reliable for compressed models has no general answer, because the available validations are platform-specific and rarely test quantized or pruned networks. Whether the rank order of compression efficiency is preserved across x86, ARM, accelerators, and GPUs is untested at the necessary breadth. The gap between nominal and realized compression has been documented in individual cases but not explained as a general property of backends and model families.

8.3 A Research Agenda for the Three Gaps

Three of the six gaps can be addressed together by a single controlled study, because they form a dependency chain rather than three separate problems: energy figures cannot be compared across compression states until the instruments are validated, and rankings cannot be compared across platforms until the proxy relationship is understood. The first is measurement-tool validation on compressed models. The corpus shows that tools disagree and that no systematic validation exists on quantized and pruned inference; a study addressing it would run a software estimator, a hardware counter, and an external meter of stated accuracy class in parallel on the same compressed models and platforms, and test whether estimator error changes systematically with compression state. The second is the mechanism behind the gap between nominal and realized compression, which is the FLOPs-energy relationship of Section 3. The roofline account explains the disconnect on individual platforms; a study addressing it would relate measured energy to operation counts and memory traffic across compression states and ask whether the memory-bandwidth mechanism predicts where the proxy fails. The third is cross-platform rank-order preservation, addressed as a protocol-matched comparison of compression rankings across platform classes.

A design of this kind has a limit that the corpus itself identifies. Energy and FLOPs are reported to correlate closely within a single architecture and to decouple as architectures are mixed, a pattern Desislavov and colleagues note from the literature they review (Desislavov et al., 2021). A study confined to one architecture family, such as convolutional networks, therefore tests the FLOPs proxy in its most favorable regime. That limitation cuts in a specific direction. If operation counts fail to rank measured energy across compression states even within one family, the result is stronger evidence against the proxy than the cross-family decoupling already reported, because it removes architectural heterogeneity as an explanation and leaves memory traffic, batch size, and backend realization, which Chang and colleagues' fixed-FLOPs results already implicate (Chang et al., 2026). If operation counts do track energy within the family, the result says nothing about whether the mechanism generalizes, and a claim about the mechanism in general would require the design to include at least one further family, such as vision transformers, whose lower arithmetic intensity gives them a stronger tendency toward memory-bound behavior (Li, Paolieri & Golubchik, 2025).

The same discipline applies to scale. Section 7.2 leaves open what scale would settle cross-platform ranking in general, and a study on a small number of platforms would not settle it either. Its contribution is methodological consistency rather than breadth: the same models in the same compression states, measured with the same instruments under the same idle-baseline, warm-up, batch-size, and reporting conventions on every platform, so that a change in ranking can be attributed to the platform rather than to differences in protocol. Such a study would establish whether rankings change under matched conditions on the platforms it covers and would provide a protocol that larger studies could reuse. It would not, on its own, establish the boundary of generalization, which remains the open question of how many and which platforms are sufficient (Section 7.4).

8.4 Why the Distinction Matters

The problem is not that energy-aware deployment is unimportant, since the pressure to reduce inference energy continues to grow. The problem is that the measurements practitioners rely on are not yet stable enough to support the decisions they must make. A team that trusts an unvalidated estimator can deploy a model that consumes more energy than the alternative it replaced. A team that optimizes a proxy metric can improve the proxy without improving energy. A team that tunes a model on one platform can find that the tuning does not transfer to another. Closing these three gaps makes each risk addressable with evidence rather than assumption: a validated measurement layer, a measured mechanism for the proxy relationship, and a known scope for cross-platform claims.

9 Conclusion

Energy has become a first-class concern in machine learning, and model compression has become the standard instrument for addressing it. This review examined the measurement layer beneath that instrument and found it less settled than its adoption suggests. Software estimators carry errors that can exceed the savings they report, hardware counters differ in domain, accuracy, and overhead, and external meters that could arbitrate between them appear in few studies. As a result, a compression claim is only as trustworthy as the measurement behind it, and the measurement is often unspecified.

The review’s central finding is a gap in measurement reliability, and its more specific contribution is an account of why the gap arises. The literature reports energy savings from compression without consistently validating the tools that measured them, distinguishing nominal from realized compression, or testing whether the savings hold across platforms. These omissions matter for mechanistic rather than procedural reasons. A compression technique reduces the work a model specifies; whether it reduces the energy the hardware spends depends on where the workload sits between compute-bound and memory-bound execution, on whether the batch size keeps the processor utilized, and on whether the backend executes the reduced-precision or sparse path at all. The same FP8 model loses on energy at small batch sizes and wins at large ones (Chung et al., 2026), pruning variants with identical FLOPs differ in measured energy by amounts whose sign depends on the execution unit (Chang et al., 2026), and quantized linear operations can run slower than their floating-point originals when the integer kernels are slower (Li, Paolieri & Golubchik, 2025).

That account changes what the field should do. The standard advice to measure on the target hardware is correct but insufficient, because it treats every deployment as a separate empirical question. The mechanisms identify which measurements carry information across deployments: the batch-size regime, the memory boundary of the platform, the backend path actually executed, and the accuracy of the reference instrument, together with the reporting items collected in Box 1 and an open measurement harness. Studies that report these can be compared and, eventually, predicted; studies that report a single energy figure for a nominal compression ratio cannot. Three pieces of work would move the evidence from the second kind to the first: validating measurement tools on compressed models, measuring realized rather than nominal compression against the mechanisms above, and comparing compression rankings across platforms under a matched protocol. Until that work is done, the field’s energy claims will rest on plausible, widely used, but not yet verified measurements, and the step from a lighter model to a lower energy cost will remain an expectation rather than a result.

References

- Alizadeh, N., & Castor, F. (2024). Green AI: A Preliminary Empirical Study on Energy Consumption in DL Models Across Different Runtime Infrastructures. arXiv Preprint arXiv:2402.13640. <https://doi.org/10.48550/arXiv.2402.13640>
- Aquino-Brítez, S., García-Sánchez, P., Ortiz, A., & Aquino-Brítez, D. (2025). Towards an Energy Consumption Index for Deep Learning Models: A Comparative Analysis of Architectures, GPUs, and Measurement Tools. *Sensors*. <https://doi.org/10.3390/s25030846>
- Bartoli, P., Veronesi, C., Giudici, A., Siorpaes, D., Trojaniello, D., & Zappa, F. (2025). Benchmarking Energy and Latency in TinyML: A Novel Method for Resource-Constrained AI. arXiv Preprint arXiv:2505.15622. <https://doi.org/10.48550/arXiv.2505.15622>
- Boumendil, A., Bechkit, W., & Benatchba, K. (2024). On-Device Deep Learning: Survey on Techniques Improving Energy Efficiency of DNNs. *IEEE Transactions on Neural Networks and Learning Systems*. <https://doi.org/10.1109/TNNLS.2024.3430028>
- Caravaca, F., Cuevas, Á., & Cuevas, R. (2025). From Prompts to Power: Measuring the Energy Footprint of LLM Inference. arXiv Preprint arXiv:2511.05597. <https://doi.org/10.48550/arXiv.2511.05597>
- Chang, W.-S., Ohk, S.-R., & Kim, Y.-J. (2026). A PMIC-Based Component-Level Power Profiling Framework for On-Device AI Optimization. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2026.3693837>
- Chung, J.-W., Ma, J. J., Wu, R., Liu, J., Kweon, O. J., Xia, Y., Wu, Z., & Chowdhury, M. (2025). The ML.ENERGY Benchmark: Toward Automated Inference Energy Measurement and Optimization. arXiv Preprint arXiv:2505.06371. <https://doi.org/10.48550/arXiv.2505.06371>
- Chung, J.-W., Wu, R., Ma, J. J., & Chowdhury, M. (2026). Where Do the Joules Go? Diagnosing Inference Energy Consumption. arXiv Preprint arXiv:2601.22076. <https://doi.org/10.48550/arXiv.2601.22076>
- Cordova-Cardenas, R., Amor, D., & Gutiérrez, Á. (2025). Edge AI in Practice: A Survey and Deployment Framework for Neural Networks on Embedded Systems. *Electronics*. <https://doi.org/10.3390/electronics14244877>
- Desislavov, R., Martínez-Plumed, F., & Hernández-Orallo, J. (2021). Compute and Energy Consumption Trends in Deep Learning Inference. arXiv Preprint arXiv:2109.05472. <https://doi.org/10.48550/arXiv.2109.05472>
- Diamond, J., & Stoico, V. (2026). What Is the Cost of Energy Monitoring? An Empirical Study on the Overhead of RAPL-Based Tools. arXiv Preprint arXiv:2604.26815. <https://doi.org/10.48550/arXiv.2604.26815>
- Dobler, S., & Lemmerich, F. (2025). Linearity-based neural network compression. arXiv Preprint arXiv:2506.21146. <https://doi.org/10.48550/arXiv.2506.21146>
- Dos Santos Ferreira, E., Lawrence, N. D., & Paleyes, A. (2026). Optimising for Energy Efficiency and Performance in Machine Learning. arXiv Preprint arXiv:2601.08991. <https://doi.org/10.48550/arXiv.2601.08991>
- Fadel Argerich, M., & Patiño-Martínez, M. (2024). Measuring and Improving the Energy Efficiency of Large Language Models Inference. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2024.3409745>
- Fanariotis, A., Orphanoudakis, T., & Fotopoulos, V. (2025). Evaluating the Energy Efficiency of NPU-Accelerated Machine Learning Inference on Embedded Microcontrollers. arXiv Preprint arXiv:2509.17533. <https://doi.org/10.48550/arXiv.2509.17533>
- Fischer, R. (2026). Ground-truthing AI energy consumption: validating CodeCarbon against external measurements. *It - Information Technology*. <https://doi.org/10.1515/itit-2025-0031>

- Francy, S., & Singh, R. (2024). Edge AI: Evaluation of Model Compression Techniques for Convolutional Neural Networks. *arXiv Preprint arXiv:2409.02134*. <https://doi.org/10.48550/arXiv.2409.02134>
- Giedra, H., Matuzevičius, D., Sledevič, T., Shubitidze, G., & Serackis, A. (2026). Benchmarking Large Language Model Inference on Limited-Resource Edge Systems. *Electronics*. <https://doi.org/10.3390/electronics15112451>
- Hernandez, D. E., Chang, J. R., & Nordling, T. E. M. (2025). Knowledge Distillation: Enhancing Neural Network Compression with Integrated Gradients. *arXiv Preprint arXiv:2503.13008*. <https://doi.org/10.48550/arXiv.2503.13008>
- Husom, E. J., Goknil, A., Astekin, M., Shar, L. K., Kåsen, A., Sen, S., Mithassel, B. A., & Soylu, A. (2025). Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency. *arXiv Preprint arXiv:2504.03360*. <https://doi.org/10.48550/arXiv.2504.03360>
- Iyer, A. C., Akshatha, N., & Suma, B. (2026). Green AI: A Comprehensive Framework for Energy-, Latency-, Memory-, and Carbon-Efficient Deep Learning Model Optimization. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2026.3714889>
- Kim, J., El Halabi, M., Ji, M., & Song, H. O. (2024). LayerMerge: Neural Network Depth Compression through Layer Pruning and Merging. *arXiv Preprint arXiv:2406.12837*. <https://doi.org/10.48550/arXiv.2406.12837>
- Lahmer, S., Khoshsirat, A., Rossi, M., & Zanella, A. (2022). Energy Consumption of Neural Networks on NVIDIA Edge Boards: an Empirical Model. *arXiv Preprint arXiv:2210.01625*. <https://doi.org/10.48550/arXiv.2210.01625>
- Lai, N., Dewi, D. A., Maidin, S. S., Xiao, W., Zhao, S., & Hu, Q. (2026). A comprehensive review of lightweight deep learning models for edge computing with future directions. *Discover Computing*. <https://doi.org/10.1007/s10791-026-10021-3>
- Li, S., Ning, X., Wang, L., Liu, T., Shi, X., Yan, S., Dai, G., Yang, H., & Wang, Y. (2024). Evaluating Quantized Large Language Models. *arXiv Preprint arXiv:2402.18158*. <https://doi.org/10.48550/arXiv.2402.18158>
- Li, Z., Paolieri, M., & Golubchik, L. (2025). A Study on Inference Latency for Vision Transformers on Mobile Devices. *arXiv Preprint arXiv:2510.25166*. <https://doi.org/10.48550/arXiv.2510.25166>
- Li, Z., Gao, T., An, Y., Chen, T., Zhang, J., Wen, Y., Liu, M., & Zhang, Q. (2025). Brain-Inspired Spiking Neural Networks for Energy-Efficient Object Detection. 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 3552–3562. <https://doi.org/10.1109/CVPR52734.2025.00336>
- Makenali, S., Rokh, B., & Azarpeyvand, A. (2025). Integrating Pruning with Quantization for Efficient Deep Neural Networks Compression. *arXiv Preprint arXiv:2509.04244*. <https://doi.org/10.48550/arXiv.2509.04244>
- Mei, L., & Stamp, M. (2025). Energy Considerations for Large Pretrained Neural Networks. *arXiv Preprint arXiv:2506.01311*. <https://doi.org/10.48550/arXiv.2506.01311>
- Mishra, A., & Lone, H. (2026). Beyond Benchmarks: Continuous Edge Inference for Fine-Grained Roadside Perception. *arXiv Preprint arXiv:2606.17241*. <https://doi.org/10.48550/arXiv.2606.17241>
- Nagel, M., Fournarakis, M., Amjad, R. A., Bondarenko, Y., van Baalen, M., & Blankevoort, T. (2021). A White Paper on Neural Network Quantization. *arXiv Preprint arXiv:2106.08295*. <https://doi.org/10.48550/arXiv.2106.08295>
- Paula, E., Soni, J., Upadhyay, H., & Lagos, L. (2025). Comparative analysis of model compression techniques for achieving carbon efficient AI. *Scientific Reports*. <https://doi.org/10.1038/s41598-025-07821-w>

- Raffin, G., & Trystram, D. (2024). Dissecting the software-based measurement of CPU energy consumption: a comparative analysis. *arXiv Preprint arXiv:2401.15985*. <https://doi.org/10.48550/arXiv.2401.15985>
- Rajput, S., & Sharma, T. (2024). Benchmarking Emerging Deep Learning Quantization Methods for Energy Efficiency. 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C). <https://doi.org/10.1109/ICSA-C63560.2024.00049>
- Rajput, S., & Sharma, T. (2026). CodeGreen: Towards Improving Precision and Portability in Software Energy Measurement. *arXiv Preprint arXiv:2603.17924*. <https://doi.org/10.48550/arXiv.2603.17924>
- Rajput, S., Widmayer, T., Shang, Z., Kechagia, M., Sarro, F., & Sharma, T. (2023). Enhancing Energy-Awareness in Deep Learning through Fine-Grained Energy Measurement. *arXiv Preprint arXiv:2308.12264*. <https://doi.org/10.48550/arXiv.2308.12264>
- Sander, J., Cohen, A., Dasari, V. R., Venable, B., & Jalaian, B. (2025). On Accelerating Edge AI: Optimizing Resource-Constrained Environments. *arXiv Preprint arXiv:2501.15014*. <https://doi.org/10.48550/arXiv.2501.15014>
- Santos, S., Ottoni, A. L. C., Borgo, R., Ferreira, D., & Nepomuceno, E. (2026). A systematic review of Green Machine Learning: practices and challenges for sustainability. *Artificial Intelligence Review*. <https://doi.org/10.1007/s10462-026-11515-8>
- Sardi, A., Alberi-Morel, M. L., Alouf, S., Giroire, F., & Moulhierac, J. (2026). WattLayer: Get Layers Right to Estimate Inference Energy of Neural Networks. *arXiv Preprint arXiv:2606.27841*. <https://doi.org/10.48550/arXiv.2606.27841>
- Schulze, J., Strassenburg, N., & Rabl, T. (2025). PQ Bench: Benchmarking Pruning and Quantization Techniques. Workshop on Data Management for End-to-End Machine Learning (DEEM '25). <https://doi.org/10.1145/3735654.3735940>
- Schwartz, R., Dodge, J., Smith, N. A., & Etzioni, O. (2019). Green AI. *arXiv Preprint arXiv:1907.10597*. <https://doi.org/10.48550/arXiv.1907.10597>
- Shalavi, N., Khoshsirat, A., Stellini, M., Zanella, A., & Rossi, M. (2023). Accurate Calibration of Power Measurements from Internal Power Sensors on NVIDIA Jetson Devices. *arXiv Preprint arXiv:2306.13107*. <https://doi.org/10.48550/arXiv.2306.13107>
- Sharma, P., & Fuerst, A. (2024). Accountable Carbon Footprints and Energy Profiling for Serverless Functions. Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC). <https://doi.org/10.1145/3698038.3698531>
- Sierra-Herrera, O. H., González Niño, M. E., Cárdenas Correa, E. F., Leon-Medina, J. X., & Pozo, F. (2026). A Reproducible Benchmarking Methodology for Machine Learning Hardware: Performance–Energy Trade-Offs from GPUs to Apple Silicon. *Algorithms*. <https://doi.org/10.3390/a19050363>
- Soltani, K., Tanwar, V. K., Gupta, A., & Das, S. K. (2025). Energy-Efficient Split Learning for Resource-Constrained Environments: A Smart Farming Solution. *arXiv Preprint arXiv:2509.02549*. <https://doi.org/10.48550/arXiv.2509.02549>
- Tschand, A., Rajan, A. T. R., Idgunji, S., Ghosh, A., Holleman, J., Kiraly, C., Ambalkar, P., Borkar, R., Chukka, R., Cockrell, T., Curtis, O., Fursin, G., Hodak, M., Kassa, H., Lokhmotov, A., Miskovic, D., Pan, Y., Manmathan, M. P., Raymond, L., ... Reddi, V. J. (2024). MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems from Microwatts to Megawatts for Sustainable AI. *arXiv Preprint arXiv:2410.12032*. <https://doi.org/10.48550/arXiv.2410.12032>
- Tu, X., Mallik, A., Chen, D., Han, K., Altintas, O., Wang, H., & Xie, J. (2023). Unveiling Energy Efficiency in Deep Learning: Measurement, Prediction, and Scoring across Edge Devices. *arXiv Preprint arXiv:2310.18329*. <https://doi.org/10.48550/arXiv.2310.18329>

- Tummalapalli, P., Arayakandy, S., Pal, R., & Kundan, K. (2026). LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load. arXiv Preprint arXiv:2603.23640. <https://doi.org/10.48550/arXiv.2603.23640>
- Xiang, G. (2026). Device-Aware Optimization of Energy-Efficient Deep Learning for Edge Deployment. IEEE Access. <https://doi.org/10.1109/ACCESS.2026.3694684>
- Xu, G., Hao, Z., Luo, Y., Hu, H., An, J., & Mao, S. (2023). DeViT: Decomposing Vision Transformers for Collaborative Inference in Edge Devices. IEEE Transactions on Mobile Computing. <https://doi.org/10.1109/TMC.2023.3315138>
- Yuan, Y., Chowdhury, M., & Talati, N. (2026). KAIROS: Stateful, Context-Aware Power-Efficient Agentic Inference Serving. arXiv Preprint arXiv:2604.16682. <https://doi.org/10.48550/arXiv.2604.16682>
- Zandonati, B., Bucagu, G., Pol, A. A., Pierini, M., Sirkin, O., & Kopetz, T. (2023). Towards Optimal Compression: Joint Pruning and Quantization. arXiv Preprint arXiv:2302.07612. <https://doi.org/10.48550/arXiv.2302.07612>
- Zou, B., Liu, N., Sun, B., Mascherin, M., Roy, D., Liu, Y., Peng, Y., Jia, N., & Chen, H. (2026). EnerInfer: Energy-Aware On-Device LLM Inference. arXiv Preprint arXiv:2606.23001. <https://doi.org/10.48550/arXiv.2606.23001>